# Distributed Systems

## Unit V: Distributed Files, Multimedia and Web Based System

**Syllabus:**

**Distributed Files:** Introduction, File System Architecture, Sun Network File System and HDFS.

**Distributed Multimedia Systems:** Characteristics of Multimedia Data, Quality of Service Management, Resource Management

**Distributed Web Based Systems:** Architecture of Traditional Web-Based Systems, Apache Web Server, Web Server Clusters, Communication by Hypertext Transfer Protocol, Synchronization, Web Proxy Caching

**Case Study:** The Global Name Service, The X.500 Directory Service, BitTorrent

- ♦ **Distributed Files**
- ❖ **Introduction**
- ❖ **File System Architecture**

**Nov_Dec_2024**

Q5) b) What are the key design issues for distributed file systems? Describe the requirements for distributed file systems.

Ans. Key Design Issues for Distributed File Systems (DFS)

A **Distributed File System (DFS)** allows users to access and share files stored on multiple computers over a network as if they were stored locally. Designing a DFS involves several important challenges and considerations.

### 1. Naming and Transparency

The DFS should provide a **uniform naming scheme** so that files can be accessed independent of their physical location.

### Types of Transparency

- **Access Transparency**
- Users access local and remote files in the same manner.
- **Location Transparency**

File name does not reveal the storage location.

- **Migration Transparency**

Files may move between servers without affecting users.

- **Replication Transparency**

Multiple copies of files should appear as a single file.

- **Concurrency Transparency**

Multiple users can access shared files simultaneously without conflicts.

- **Failure Transparency**

System should continue operating despite failures.

## 2. File Sharing Semantics

Defines how updates made by one user become visible to others.

### Common Semantics

- **UNIX Semantics**

Changes are immediately visible to all users.

- **Session Semantics**

Changes become visible after the file is closed.

- **Immutable Shared Files**

Files cannot be modified once created.

- **Transactional Semantics**

File operations are treated as transactions.

## 3. Caching

Caching improves performance by storing frequently accessed data locally.

### Design Considerations

- Where to cache:

    - Client memory
    - Client disk
    - Server cache
- Cache consistency maintenance

### Advantages

- Reduced network traffic
- Faster access
- Better scalability

## 4. Replication

Multiple copies of files may be stored on different servers.

### Goals

- Improve availability
- Increase reliability
- Reduce access latency

### Challenges

- Maintaining consistency among replicas
- Synchronizing updates

## 5. Fault Tolerance and Recovery

DFS must continue operating even when:

- Servers fail
- Network links fail
- Clients crash

### Mechanisms

- Redundant servers
- Backup and recovery
- Logging and checkpointing

- Replication

### 6. Scalability

The system should support:

- Large numbers of users
- Large file collections
- Geographically distributed nodes

#### Important Techniques

- Distributed naming
- Load balancing
- Decentralized control

### 7. Security

DFS operates over networks, making security essential.

#### Security Requirements

- Authentication
- Authorization
- Access control
- Encryption
- Secure communication

### 8. Concurrency Control

Many users may access the same file simultaneously.

#### Problems

- Lost updates
- Inconsistent reads

#### Solutions

- File locking

- Mutual exclusion
- Version control

### 9. Heterogeneity

DFS should support different:

- Operating systems
- Hardware platforms
- Communication protocols

Example:

- Windows clients accessing Linux file servers.

### 10. Performance

Performance is a major concern because communication over a network is slower than local disk access.

#### Performance Optimization

- Caching
- Replication
- Load balancing
- Efficient communication protocols

## Requirements for Distributed File Systems

A good Distributed File System should satisfy the following requirements:

### 1. Transparency

Users should not be aware that files are distributed across multiple machines.

Includes:

- Access transparency
- Location transparency
- Replication transparency

- Failure transparency

### 2. High Availability

Files should remain accessible even if:

- A server crashes
- Network problems occur

Achieved through:

- Replication
- Backup servers
- Fault recovery mechanisms

### 3. Reliability

The system must preserve data integrity and prevent data loss.

#### Methods

- Journaling
- Checkpointing
- Stable storage

### 4. Scalability

DFS should continue functioning efficiently as:

- Users increase
- Servers increase
- Data volume grows

### 5. Performance

The system should provide:

- Fast file access
- Low response time
- Efficient network usage

### 6. Consistency

All users should see a consistent view of shared data.

Consistency mechanisms include:

- Cache consistency protocols
- File locking
- Synchronization techniques

### 7. Security

The system must prevent unauthorized access.

#### Security Features

- User authentication
- Access permissions
- Encryption

### 8. Heterogeneity Support

The DFS should support different:

- Devices
- Operating systems
- Networks

### 9. Mobility Support

Users should be able to access files from different locations and devices.

### 10. Easy Administration

System management should be simple.

Includes:

- Centralized management
- Automated backups
- Monitoring tools

## Conclusion

Designing a Distributed File System involves balancing:

- Performance
- Scalability
- Reliability
- Security
- Transparency

An effective DFS provides users with seamless and efficient access to distributed data while handling failures, concurrency, and network complexities transparently.

❖ **Sun Network File System**

**May_Jun_2023**

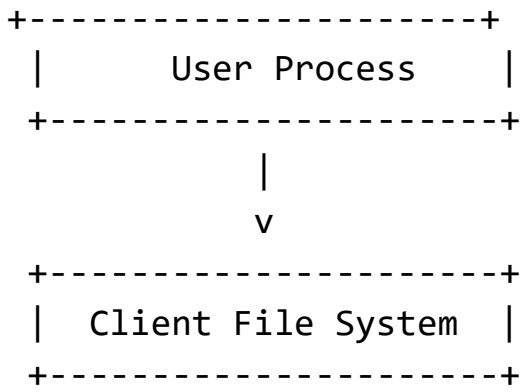
Q5) a) Describe the architecture of Sun Network File System in details.

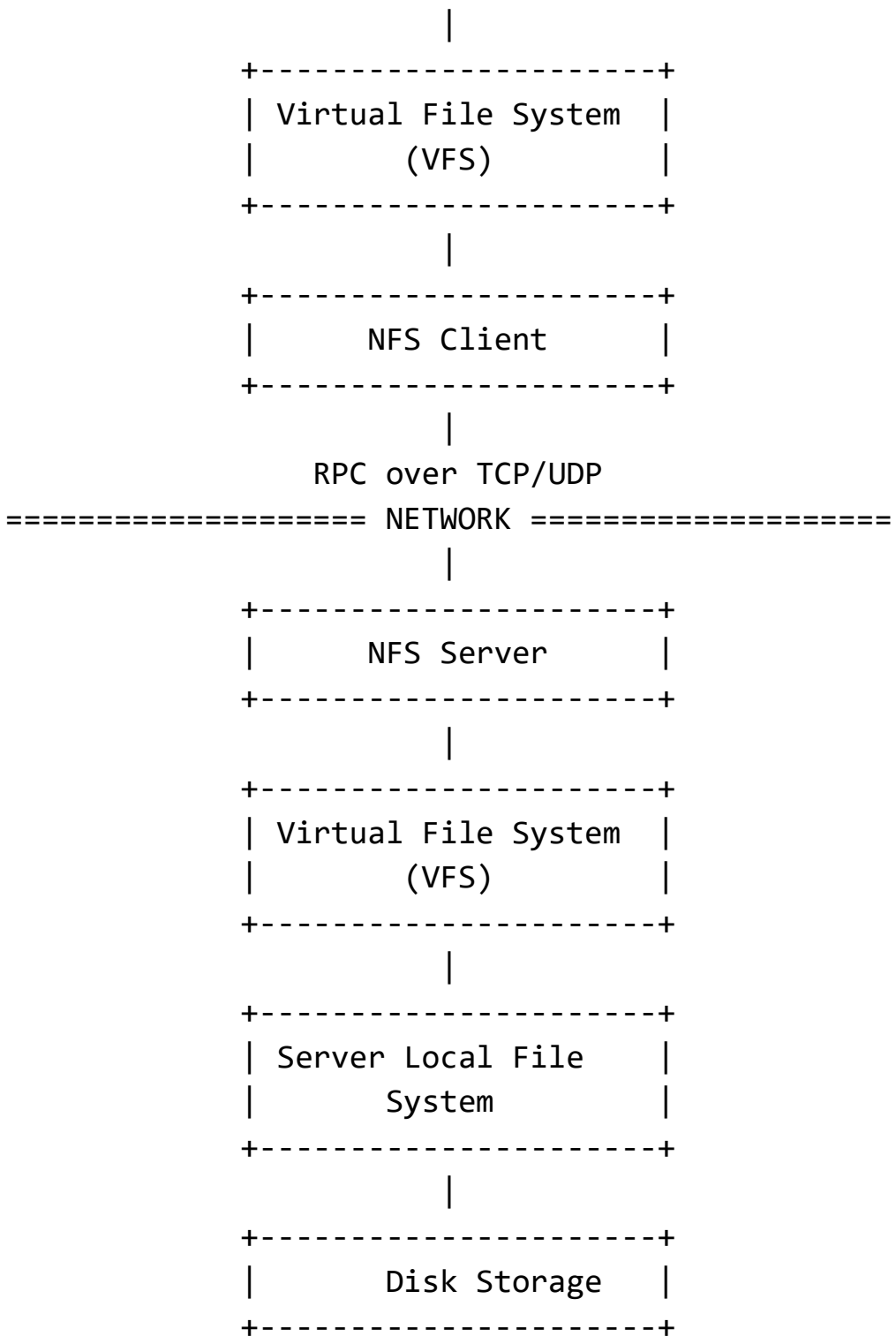
Ans. Architecture of Sun Network File System (NFS)

The Sun Network File System is a distributed file system developed by Sun Microsystems that allows a client computer to access remote files over a network as if they were stored locally.

NFS follows a **client-server architecture** and is based on the **Remote Procedure Call (RPC)** mechanism.

## NFS Architecture Diagram




Virtual File System
(VFS)
NFS Client
RPC over TCP/UDP
NETWORK
NFS Server
Virtual File System
(VFS)
Server Local File
System
Disk Storage

## Components of NFS Architecture

### 1. User Process

- User applications issue normal file operations such as:
  - open()
  - read()
  - write()
  - close()
- Users are unaware whether the file is local or remote.

### 2. Client File System

The client operating system receives file-related system calls from user programs.

#### Functions

- Interprets file operations
- Passes requests to VFS

### 3. Virtual File System (VFS)

VFS provides a common interface for different file systems.

#### Role

- Distinguishes between:
  - Local files
  - Remote NFS files

#### Advantages

- Transparency
- Supports multiple file systems simultaneously

### 4. NFS Client Module

Responsible for handling remote file access.

#### Functions

- Converts file operations into RPC requests

- Sends requests to the server
- Receives server responses

### Example

A read request becomes an RPC message sent across the network.

### 5. RPC Layer

NFS uses **Remote Procedure Calls (RPC)** for communication.

### Features

- Hides network communication details
- Supports communication using:
    - TCP
    - UDP

### Working

- Client calls remote procedures on the server as if local.

### 6. Network

The network transfers RPC messages between client and server.

### 7. NFS Server Module

Runs on the server machine.

### Functions

- Receives RPC requests
- Performs requested file operations
- Returns results to clients

### 8. Server VFS

Provides abstraction for local file systems on the server.

### 9. Local File System

The actual file system managing disk storage.

Examples:

- UFS
- ext4
- NTFS

### 10. Disk Storage

Stores physical file data.

## Working of NFS

### Step-by-Step Operation

### Step 1: File Request

A user process requests access to a remote file.

### Step 2: VFS Check

The client VFS determines the file is remote.

### Step 3: RPC Generation

The NFS client converts the request into an RPC call.

### Step 4: Network Communication

The RPC request is sent to the NFS server.

### Step 5: Server Processing

The NFS server performs the required operation on local storage.

### Step 6: Reply

Results are sent back to the client.

### Step 7: User Access

The client application receives the data transparently.

## Important Features of NFS

### 1. Stateless Server

Traditional NFS servers are stateless.

### Meaning

The server does not maintain client session information.

### Advantages

- Easier recovery after crashes
- Simpler server design

## 2. File Handles

NFS identifies files using **file handles**.

A file handle contains:

- File system identifier
- Inode number
- Generation number

## 3. Mount Protocol

Clients must mount remote directories before accessing them.

Example:

```
mount server:/shared /mnt/shared
```

### Advantages of NFS

- Transparent remote file access
- Platform independence
- Centralized file sharing
- Easy administration
- Scalability

### Limitations of NFS

- Performance depends on network speed
- Security vulnerabilities in older versions
- Cache consistency problems
- Stateless design may reduce efficiency

## Conclusion

The Sun Network File System architecture uses a client-server model with RPC communication and VFS abstraction to provide transparent remote file access. Its modular design, stateless operation, and portability made it one of the most widely used distributed file systems in networked environments.

- ❖ **HDFS.**

**May_Jun_2024**

Q5) a) Discuss the master-slave architecture of Hadoop Distributed File System along with functions of its key components.

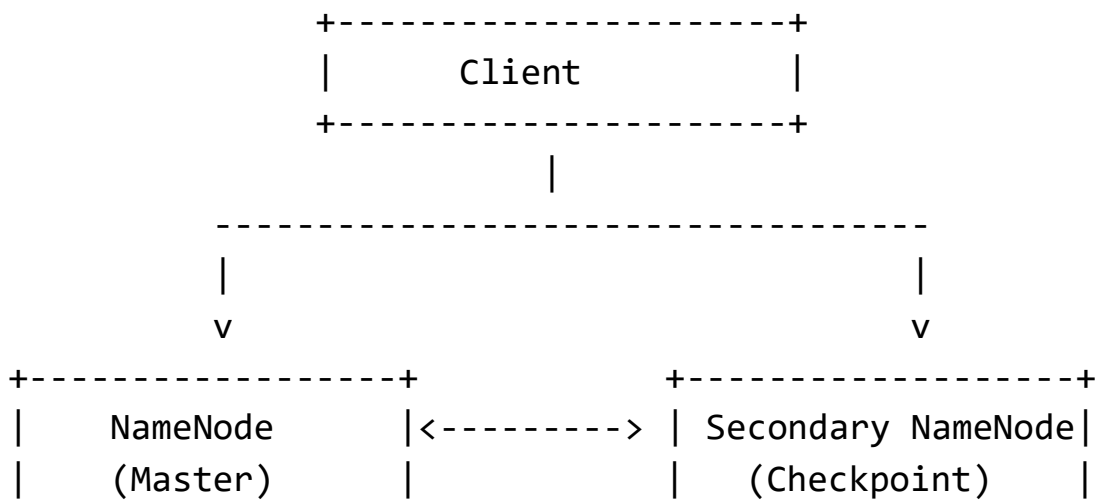
Ans. Master-Slave Architecture of Hadoop Distributed File System (HDFS)

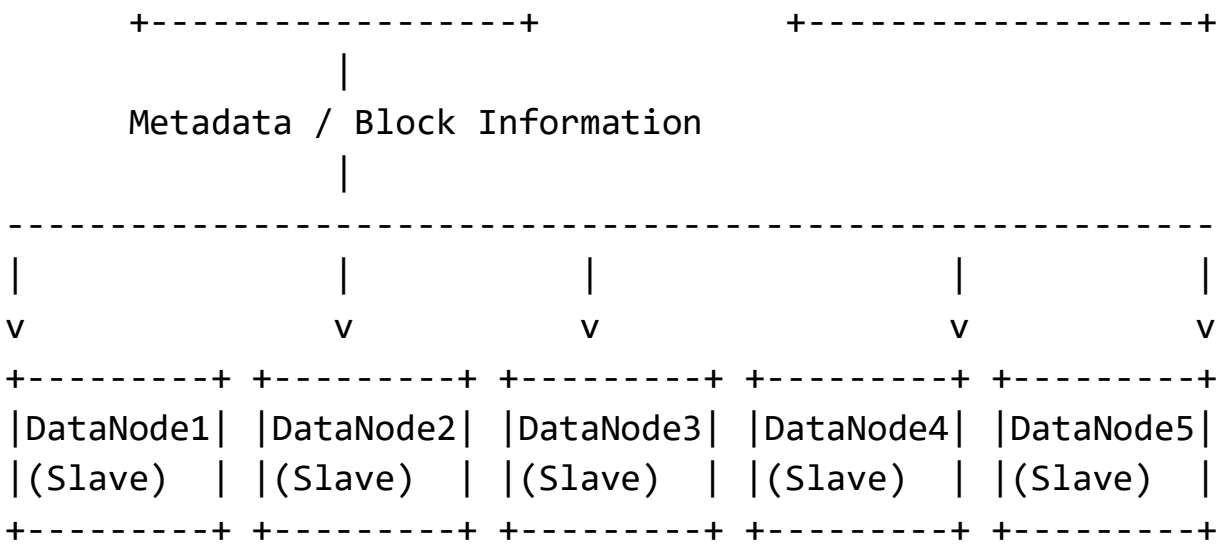
The Hadoop Distributed File System is a distributed file system designed to store very large files across multiple machines in a cluster. It follows a **master-slave architecture** for reliable and scalable storage.

HDFS is a part of Apache Hadoop and is optimized for:

- High throughput
- Fault tolerance
- Large-scale data storage

## HDFS Master-Slave Architecture Diagram






## Key Components of HDFS

### 1. NameNode (Master Node)

The **NameNode** is the central component of HDFS.

#### Main Responsibilities

- Manages the file system namespace
- Maintains metadata about files and directories
- Keeps track of:
    - File names
    - Permissions
    - File locations
    - Block information

#### Functions

- Decides where blocks are stored
- Handles client requests
- Controls access to files
- Monitors DataNodes

#### Metadata Stored by NameNode

- File hierarchy

- Mapping of files to blocks
- Mapping of blocks to DataNodes

### Important Characteristics

- Only one active NameNode in traditional HDFS
- Stores metadata in memory for fast access

### Files Maintained

- **FsImage** → snapshot of file system metadata
- **EditLog** → recent changes to metadata

## 2. DataNode (Slave Nodes)

DataNodes are worker nodes responsible for actual data storage.

### Main Responsibilities

- Store file blocks
- Read and write data blocks
- Send status reports to NameNode

### Functions

- Perform block creation, deletion, replication
- Serve read/write requests from clients
- Periodically send:
    - Heartbeats
    - Block reports

### Block Storage

Files are divided into large blocks (typically 128 MB or more).

Example:

```
File A
 ├── Block 1 → DataNode1
 ├── Block 2 → DataNode3
```

```
└── Block 3 → DataNode5
```

### 3. Secondary NameNode

Despite its name, it is **not a backup NameNode**.

#### Purpose

Helps the NameNode by periodically merging:

- FsImage
- EditLog

#### Functions

- Creates checkpoints
- Reduces EditLog size
- Improves NameNode recovery time

#### Important Note

If the NameNode fails, the Secondary NameNode does not automatically replace it.

### 4. Client

The client interacts with HDFS to:

- Read files
- Write files
- Delete files

#### Client Interaction

- Contacts NameNode for metadata
- Directly communicates with DataNodes for actual data transfer

## Working of HDFS Architecture

### File Write Operation

#### Step 1: Client Request

Client requests the NameNode to create a file.

#### Step 2: Metadata Handling

NameNode checks permissions and creates metadata entries.

#### Step 3: Block Allocation

NameNode selects DataNodes for storing blocks.

#### Step 4: Data Storage

Client sends data directly to DataNodes.

#### Step 5: Replication

Blocks are replicated to multiple DataNodes.

### File Read Operation

#### Step 1: Metadata Request

Client asks NameNode for block locations.

#### Step 2: Block Information

NameNode returns DataNode addresses.

#### Step 3: Data Access

Client directly reads blocks from DataNodes.

### Block Replication in HDFS

HDFS maintains multiple copies of each block.

#### Default Replication Factor

Usually:

```
Replication Factor = 3
```

### Advantages

- Fault tolerance
- High availability
- Reliability

Example:

```
Block A:
  Copy 1 → DataNode1
  Copy 2 → DataNode3
  Copy 3 → DataNode5
```

## Heartbeat and Block Reports

### Heartbeat

DataNodes periodically send heartbeat messages to NameNode.

### Purpose

Indicates that the DataNode is alive.

If heartbeat stops:

- NameNode assumes node failure.

### Block Report

Contains information about:

- Stored blocks
- Block status

Sent periodically to NameNode.

## Advantages of HDFS Master-Slave Architecture

### 1. Scalability

Supports thousands of nodes and massive datasets.

### 2. Fault Tolerance

Replication ensures data availability despite failures.

### 3. High Throughput

Optimized for large data processing.

### 4. Cost Effective

Runs on commodity hardware.

### 5. Reliability

Automatic recovery from node failures.

## Limitations of HDFS

### 1. Single Point of Failure

Traditional NameNode architecture may fail if NameNode crashes.

### 2. Not Suitable for Small Files

Large metadata overhead for many small files.

### 3. High Latency

Not ideal for real-time applications.

## Conclusion

The Hadoop Distributed File System uses a master-slave architecture where:

- The **NameNode** manages metadata,
- **DataNodes** store actual data blocks,
- The **Secondary NameNode** performs checkpointing.

This architecture provides scalability, reliability, and fault tolerance for large-scale distributed data storage systems.

- ♦ **Distributed Multimedia Systems**
- ❖ **Characteristics of Multimedia Data**

## Characteristics of Multimedia Data

Multimedia data refers to the integration of multiple forms of information such as:

- Text
- Audio
- Images
- Graphics
- Animation
- Video

Multimedia data used in distributed systems and multimedia applications has several unique characteristics that distinguish it from traditional data.

### 1. Large Volume of Data

Multimedia files usually require huge storage space.

#### Examples

- High-definition videos
- Audio recordings
- Image collections

#### Reason

Video and audio contain large amounts of continuous information.

#### Example

```
1 hour HD video
≈ several GB of storage
```

#### Impact

- Requires large storage capacity
- High bandwidth needed for transmission

### 2. Time Dependency (Temporal Nature)

Many multimedia data types are time-dependent.

### Examples

- Audio streams
- Video streams
- Animation

The data must be delivered and presented at the correct time.

### Importance

Proper timing ensures:

- Smooth playback
- Synchronization
- Real-time interaction

## 3. Continuous Media

Multimedia data is often continuous in nature.

### Continuous Media

Data generated continuously over time.

Examples:

- Audio
- Video

### Non-Continuous Media

Examples:

- Text
- Images

## 4. Real-Time Requirements

Many multimedia applications require real-time communication.

### Applications

- Video conferencing

- Online gaming
- Live streaming
- VoIP

### Requirement

Data must arrive within strict deadlines.

### Effects of Delay

- Audio gaps
- Video freezing
- Poor user experience

## 5. High Bandwidth Requirement

Multimedia applications consume significant network bandwidth.

### Example

| Data Type | Bandwidth Requirement |
|---|---|
| Text | Low |
| Audio | Medium |
| HD Video | Very High |

### Impact

Efficient compression and transmission techniques are necessary.

## 6. Synchronization Requirement

Different multimedia components must remain synchronized.

### Example

Audio and video in a movie must match.

### Types of Synchronization

#### a) Intra-media Synchronization

Synchronization within a single media stream.

Example:

- Correct order of video frames

#### b) Inter-media Synchronization

Synchronization between multiple media types.

Example:

- Lip synchronization between audio and video

### 7. Loss Tolerance

Some multimedia applications can tolerate limited data loss.

#### Example

Small packet loss in video streaming may not be noticeable.

Unlike text data:

- Minor multimedia errors may still allow acceptable playback.

### 8. QoS Sensitivity

Multimedia data is highly sensitive to:

- Delay
- Jitter
- Packet loss
- Bandwidth fluctuations

#### Therefore

Quality of Service (QoS) mechanisms are essential.

### 9. Compression Requirement

Because multimedia data is very large, compression is necessary.

### Types of Compression

#### a) Lossless Compression

No information loss.

Examples:

- PNG
- FLAC

#### b) Lossy Compression

Some information removed for smaller size.

Examples:

- MP3
- JPEG
- MPEG

### 10. Heterogeneity

Multimedia systems operate across different:

- Devices
- Networks
- Operating systems
- File formats

### Examples

- Mobile phones
- PCs
- Smart TVs

### 11. Interactive Nature

Many multimedia applications support user interaction.

#### Examples

- Video games
- Interactive learning
- Virtual reality

Users can:

- Pause
- Rewind
- Control playback

### 12. Storage and Retrieval Complexity

Efficient indexing and retrieval of multimedia data is difficult.

#### Challenges

- Searching videos
- Managing large media databases
- Content-based retrieval

### 13. Error Resilience Requirement

Multimedia systems should continue functioning despite:

- Packet loss
- Network congestion
- Temporary failures

#### Techniques

- Error correction
- Retransmission
- Buffering

## 14. Streaming Support

Multimedia data is often transmitted using streaming techniques.

### Streaming Characteristics

- Playback begins before full download
- Continuous delivery of media packets

### Examples

- YouTube
- Netflix
- Spotify

## 15. Multiple Data Formats

Multimedia data exists in many formats.

### Examples

**Examples**

| Media Type | Formats |
| --- | --- |
| Image | JPEG, PNG, GIF |
| Audio | MP3, WAV |
| Video | MP4, AVI, MPEG |

## Summary Table

| Characteristic | Description |
|---|---|
| Large Volume | Requires huge storage |
| Time Dependency | Sensitive to timing |
| Continuous Media | Continuous data generation |
| Real-Time Nature | Requires timely delivery |
| High Bandwidth | Needs fast networks |
| Synchronization | Audio-video coordination |
| Loss Tolerance | Minor loss acceptable |
| QoS Sensitivity | Sensitive to delay/loss |
| Compression | Reduces file size |
| Interactivity | Supports user control |

## Conclusion

Multimedia data possesses unique characteristics such as large size, time dependency, synchronization needs, and real-time requirements. These characteristics make multimedia systems more complex than traditional systems and require specialized techniques for storage, transmission, compression, synchronization, and Quality of Service management in distributed environments.

- ❖ **Quality of Service Management**

**May_Jun_2024**

Q6) a) Explain the Bandwidth, Latency and Loss rate parameters with respect to multimedia stream. Explain the QoS negotiation procedure and admission control scheme for distributed multimedia application.

Ans. Multimedia Stream Parameters

In distributed multimedia applications such as video conferencing, online streaming, VoIP, and real-time gaming, the quality of communication depends heavily on network performance parameters. The most important parameters are:

- Bandwidth
- Latency
- Loss Rate

These parameters directly affect the **Quality of Service (QoS)** provided to multimedia applications.

## 1. Bandwidth

### Definition

Bandwidth is the amount of data that can be transmitted through a network in a given amount of time.

It is usually measured in:

- bits per second (bps)
- Kbps
- Mbps
- Gbps

### In Multimedia Streaming

Bandwidth determines:

- Audio/video quality
- Resolution
- Frame rate
- Smooth playback

### Example

- HD video requires higher bandwidth than standard video.

- Video conferencing needs continuous bandwidth for uninterrupted communication.

### Effects of Low Bandwidth

- Buffering
- Reduced video quality
- Audio distortion
- Frame drops

### Example

```
Video Stream Requirement:
4 Mbps
Available Network Bandwidth:
1 Mbps
Result:
Poor quality and buffering
```

## 2. Latency

### Definition

Latency is the delay between sending and receiving data.

Measured in:

- milliseconds (ms)

### Types of Delay

#### a) Transmission Delay

Time required to place data onto the communication link.

#### b) Propagation Delay

Time taken by signals to travel through the medium.

#### c) Processing Delay

Time routers/systems take to process packets.

### d) Queuing Delay

Waiting time in network queues.

### In Multimedia Streaming

Latency is extremely important for:

- Video conferencing
- Online gaming
- Live streaming
- VoIP calls

### Effects of High Latency

- Audio-video synchronization problems
- Delay in conversations
- Poor interactive experience

### Example

```
Latency < 150 ms:
Good video conferencing quality
Latency > 400 ms:
Noticeable conversation delay
```

## 3. Loss Rate

### Definition

Loss rate refers to the percentage of packets lost during transmission.

$$\text{Loss Rate} = \frac{\text{Packets Lost}}{\text{Packets Sent}} \times 100$$

### Causes of Packet Loss

- Network congestion
- Faulty hardware
- Buffer overflow
- Wireless interference

## Effects on Multimedia

### Audio

- Distortion
- Missing voice packets

### Video

- Pixelation
- Frozen frames
- Video artifacts

### Acceptable Loss Levels

| Application | Acceptable Loss |
|---|---|
| Voice | < 1% |
| Video Streaming | < 2–5% |
| File Transfer | Nearly 0% |

## Quality of Service (QoS)

### Definition

Quality of Service (QoS) refers to the network's ability to provide guaranteed performance for multimedia applications.

QoS ensures:

- Required bandwidth
- Controlled latency
- Minimal packet loss
- Jitter control

### QoS Negotiation Procedure

QoS negotiation is the process through which multimedia applications request network resources before data transmission begins.

## Steps in QoS Negotiation

### Step 1: QoS Request by Client

The client specifies its multimedia requirements such as:

- Required bandwidth
- Maximum latency
- Maximum packet loss
- Jitter limits

### Example

```
Video Conference Requirements:
Bandwidth = 5 Mbps
Latency < 100 ms
Loss Rate < 1%
```

### Step 2: Resource Availability Check

The network or server checks whether requested resources are available.

It verifies:

- Available bandwidth
- Network congestion status
- Buffer availability

### Step 3: QoS Negotiation

If exact requirements cannot be met:

- Alternative QoS levels are proposed.

### Example

```
Requested:
5 Mbps
Available:
3 Mbps
```

```
Negotiated:
Reduced video resolution
```

### Step 4: Resource Reservation

After agreement:

- Network resources are reserved.
- Priorities may be assigned.

Protocols such as:

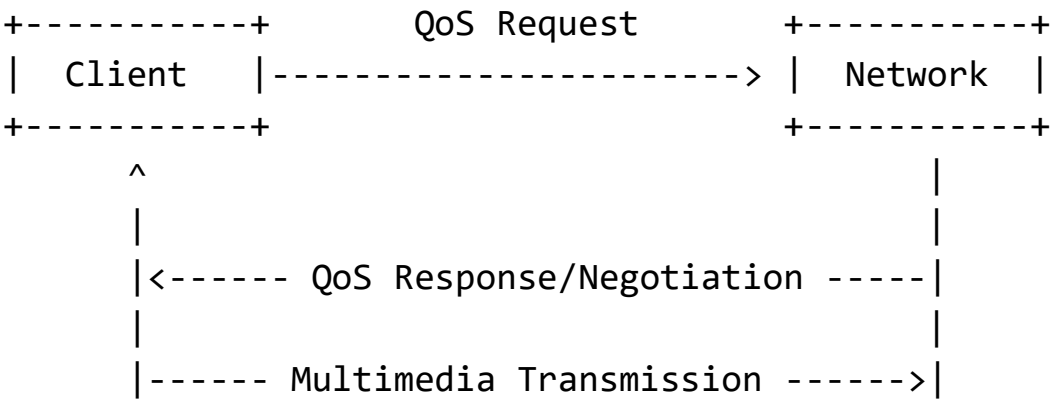
- RSVP (Resource Reservation Protocol)

may be used.

### Step 5: Multimedia Transmission

Data transmission begins using the negotiated QoS parameters.

### QoS Negotiation Diagram




### Admission Control Scheme

#### Definition

Admission control determines whether a new multimedia stream should be accepted or rejected based on resource availability.

Its purpose is to:

- Prevent network overload
- Maintain QoS for existing users

## Working of Admission Control

### Step 1: New Stream Request

A multimedia application requests network access with QoS requirements.

### Step 2: Resource Evaluation

The system checks:

- Current bandwidth usage
- Buffer capacity
- Delay constraints
- Existing QoS commitments

### Step 3: Decision Making

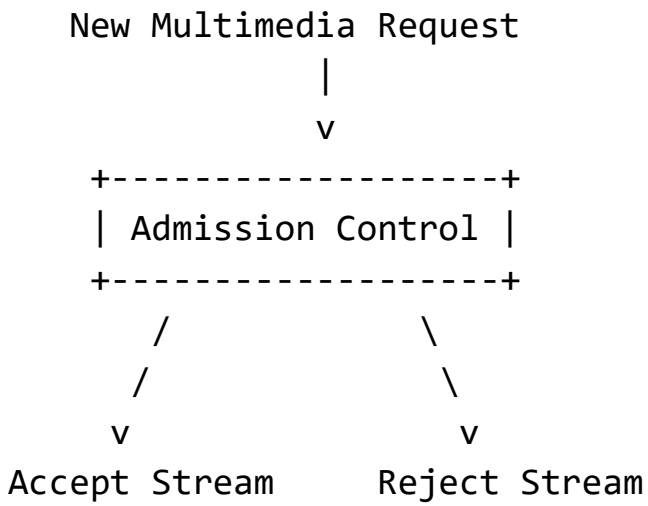
### Case 1: Resources Available

- Request is accepted.

### Case 2: Resources Insufficient

- Request is rejected or renegotiated.

### Admission Control Diagram



## Types of Admission Control

### 1. Parameter-Based Admission Control

Decision based on:

- Fixed thresholds
- Mathematical calculations

### 2. Measurement-Based Admission Control

Uses real-time network measurements such as:

- Traffic load
- Delay
- Congestion

## Importance of Admission Control

- Prevents congestion
- Guarantees QoS
- Improves reliability
- Ensures fairness among users

## Relationship Between QoS Parameters

| Parameter | Effect on Multimedia |
|---|---|
| Bandwidth | Determines quality and smoothness |
| Latency | Affects interactivity |
| Loss Rate | Impacts audio/video clarity |

## Applications of QoS in Multimedia

- Video conferencing
- Live streaming
- IPTV
- Online gaming

- Telemedicine
- Distance learning

## Conclusion

In distributed multimedia systems, bandwidth, latency, and loss rate are critical QoS parameters that determine communication quality. QoS negotiation allows applications and networks to agree on suitable service levels, while admission control ensures that only manageable multimedia streams are accepted, thereby maintaining reliable and efficient multimedia communication.

**Nov_Dec_2023**

Q5) b) Why Quality of Service Management is important in Distributed Multimedia Systems? Describe QoS manager responsibilities using suitable graphical representation.

Ans. Importance of Quality of Service (QoS) Management in Distributed Multimedia Systems

A **Distributed Multimedia System** delivers multimedia content such as:

- Audio
- Video
- Animation
- Real-time conferencing

over computer networks.

Applications like:

- Video conferencing
- Online gaming
- IPTV
- Telemedicine
- Live streaming

require guaranteed performance for smooth operation.

This guaranteed performance is called **Quality of Service (QoS).**

### What is QoS?

Quality of Service (QoS) refers to the ability of a system or network to provide predictable and guaranteed service performance for multimedia applications.

QoS ensures:

- Sufficient bandwidth
- Low latency
- Controlled jitter
- Minimal packet loss
- Synchronization

## Importance of QoS Management in Distributed Multimedia Systems

### 1. Ensures Real-Time Communication

Multimedia applications are often real-time in nature.

#### Examples

- Video conferencing
- VoIP
- Online gaming

QoS management guarantees timely delivery of multimedia packets.

#### Without QoS

- Delayed audio
- Frozen video
- Communication gaps

### 2. Controls Delay (Latency)

High delay affects interactive multimedia applications.

### Effects

- Delayed conversations
- Poor gaming response
- Unsynchronized communication

QoS mechanisms maintain acceptable delay limits.

## 3. Reduces Jitter

### Jitter

Variation in packet arrival time.

### Problems Caused

- Choppy audio
- Jerky video playback

QoS management smooths packet delivery.

## 4. Minimizes Packet Loss

Packet loss degrades multimedia quality.

### Effects

- Missing audio
- Pixelated video
- Frame drops

QoS ensures reliable packet transmission.

## 5. Provides Bandwidth Guarantee

Multimedia applications require large bandwidth.

QoS reserves sufficient network resources to:

- Prevent congestion
- Maintain media quality

### 6. Maintains Synchronization

Audio and video streams must remain synchronized.

#### Example

Lip synchronization in video conferencing.

QoS management ensures coordinated delivery.

### 7. Improves User Experience

Good QoS leads to:

- Smooth playback
- Better interactivity
- Higher reliability

This improves overall Quality of Experience (QoE).

### 8. Supports Resource Allocation

QoS helps efficiently allocate:

- CPU
- Memory
- Buffers
- Network bandwidth

among multiple multimedia streams.

### 9. Prevents Network Congestion

QoS mechanisms:

- Prioritize important traffic
- Control overload conditions
- Maintain stable performance

### 10. Enables Scalability

QoS management allows distributed multimedia systems to support:

- Many users
- Large-scale multimedia traffic
- Multiple concurrent streams

### QoS Parameters

Important QoS parameters include:

| Parameter | Meaning |
|---|---|
| Bandwidth | Data transfer capacity |
| Latency | Transmission delay |
| Jitter | Variation in delay |
| Loss Rate | Packet loss percentage |
| Reliability | Correct data delivery |

## QoS Manager

### Definition

A **QoS Manager** is a component responsible for monitoring, controlling, and managing multimedia resources to maintain required QoS levels.

It acts as a coordinator between:

- Applications
- Network
- Operating system
- Resource managers

### Responsibilities of QoS Manager

#### 1. QoS Negotiation

The QoS manager negotiates QoS requirements between:

- Client
- Server
- Network

### Example

```
Requested:
Bandwidth = 5 Mbps
Available:
3 Mbps
Negotiated:
Lower video resolution
```

### 2. Resource Reservation

Reserves required resources such as:

- Network bandwidth
- CPU time
- Buffers
- Storage

to satisfy multimedia requirements.

### 3. Admission Control

Determines whether a new multimedia stream can be accepted.

### Decision Based On

- Available resources
- Existing system load
- QoS commitments

If resources are insufficient:

- Request may be rejected.

### 4. Monitoring QoS

Continuously monitors:

- Delay
- Packet loss
- Jitter
- Bandwidth utilization

to ensure QoS guarantees are maintained.

### 5. QoS Adaptation

Adjusts multimedia quality dynamically during congestion.

#### Example

- Reduce video resolution
- Lower frame rate
- Compress data further

### 6. Scheduling and Prioritization

Schedules multimedia traffic based on priority.

#### Example

Real-time voice packets receive higher priority than email traffic.

### 7. Synchronization Control

Ensures synchronization among multimedia streams.

#### Example

Matching audio with video playback.

### 8. Fault Handling and Recovery

Detects failures and takes corrective actions such as:

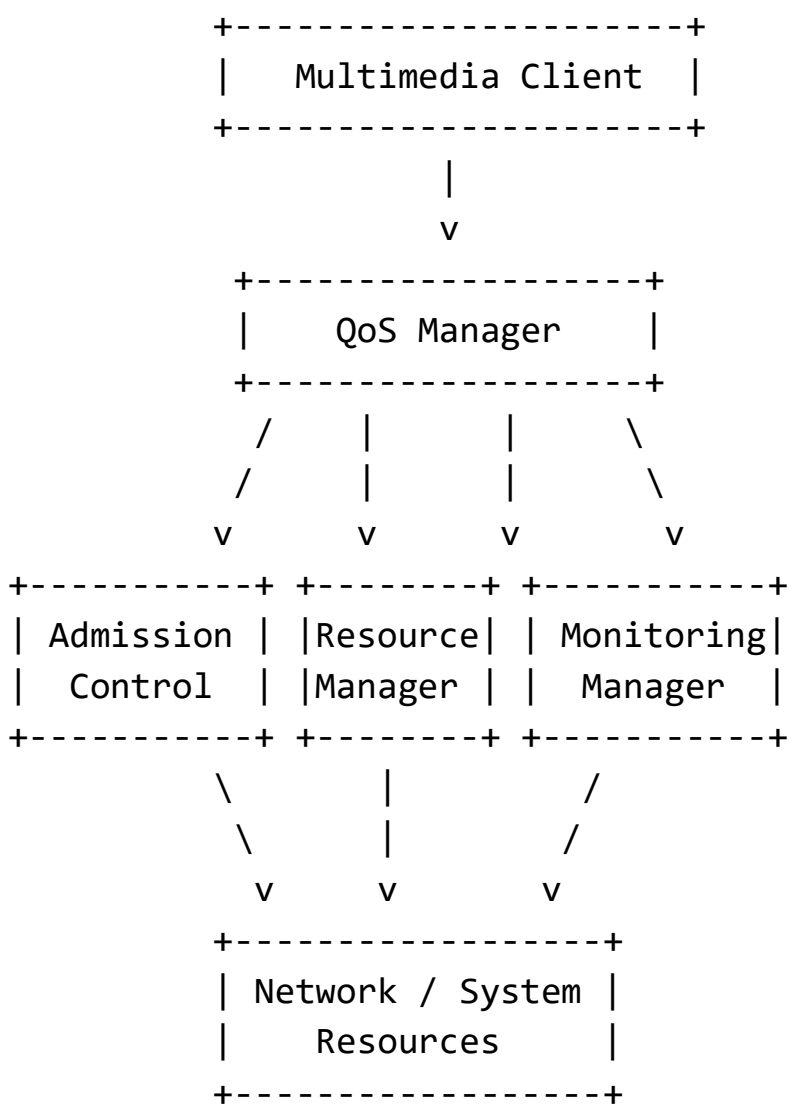
- Resource reallocation
- Stream rerouting
- Error recovery

### 9. Traffic Management

Controls multimedia traffic to avoid:

- Congestion
- Overloading
- Performance degradation

### Graphical Representation of QoS Manager



## Working of QoS Manager

### Step 1: Application Request

The multimedia application specifies QoS requirements.

### Step 2: QoS Negotiation

QoS manager negotiates available resources.

### Step 3: Admission Control

Checks whether sufficient resources are available.

### Step 4: Resource Allocation

Allocates bandwidth, CPU, and buffers.

### Step 5: Continuous Monitoring

Tracks QoS performance during transmission.

### Step 6: Dynamic Adjustment

Adjusts resource allocation if network conditions change.

## Advantages of QoS Management

- Smooth multimedia playback
- Reduced delay and jitter
- Efficient resource utilization
- Better reliability
- Improved scalability
- Enhanced user satisfaction

## Conclusion

Quality of Service management is essential in distributed multimedia systems because multimedia applications are highly sensitive to delay, bandwidth fluctuations, packet loss, and synchronization problems. A QoS manager ensures efficient resource allocation, admission control, monitoring,

adaptation, and synchronization to maintain reliable and high-quality multimedia communication.

❖ **Resource Management**

## Resource Management in Distributed Systems

### Definition

Resource Management is the process of efficiently allocating, controlling, monitoring, and scheduling system resources among multiple users, applications, and processes in a distributed system.

Resources may include:

- CPU
- Memory
- Storage
- Network bandwidth
- Printers
- Multimedia devices
- Databases

The main goal is to ensure:

- Efficient utilization
- Fair allocation
- High performance
- Reliability
- Scalability

### Objectives of Resource Management

1. Efficient resource utilization
2. Fair sharing among users
3. Avoidance of deadlocks and starvation
4. High system performance

5. Load balancing
6. Fault tolerance
7. QoS maintenance
8. Scalability support

## Types of Resources

| Resource Type | Examples |
|---|---|
| Processing Resources | CPU, GPU |
| Storage Resources | Disk, SSD, Databases |
| Communication Resources | Bandwidth, Routers |
| Peripheral Resources | Printers, Scanners |
| Multimedia Resources | Audio/Video devices |

## Components of Resource Management

### 1. Resource Allocation

Allocates resources to processes or users according to requirements.

#### Example

- Assigning CPU time to processes
- Allocating memory to applications

#### Goals

- Maximum utilization
- Fairness
- Reduced waiting time

### 2. Resource Scheduling

Determines:

- Which process gets resources

- For how long
- In what order

### Scheduling Types

- CPU scheduling
- Network scheduling
- Disk scheduling

### Common Algorithms

- FCFS
- Round Robin
- Priority Scheduling

## 3. Load Balancing

Distributes workload evenly among multiple nodes.

### Purpose

- Prevent overloading
- Improve performance
- Increase throughput

### Example

Cloud servers distributing user requests.

## 4. Resource Monitoring

Tracks resource usage continuously.

### Monitored Parameters

- CPU utilization
- Memory usage
- Network traffic
- Disk usage

### Benefits

- Detect bottlenecks
- Improve optimization

## 5. Resource Reservation

Reserves resources in advance for critical applications.

### Example

Video conferencing reserving bandwidth.

### Used In

- Multimedia systems
- Real-time systems

## 6. Admission Control

Determines whether a new request can be accepted.

### Purpose

Prevent system overload.

### Decision Based On

- Available resources
- Current system load

## 7. Deadlock Handling

Deadlock occurs when processes wait indefinitely for resources.

### Methods

- Prevention
- Avoidance
- Detection
- Recovery

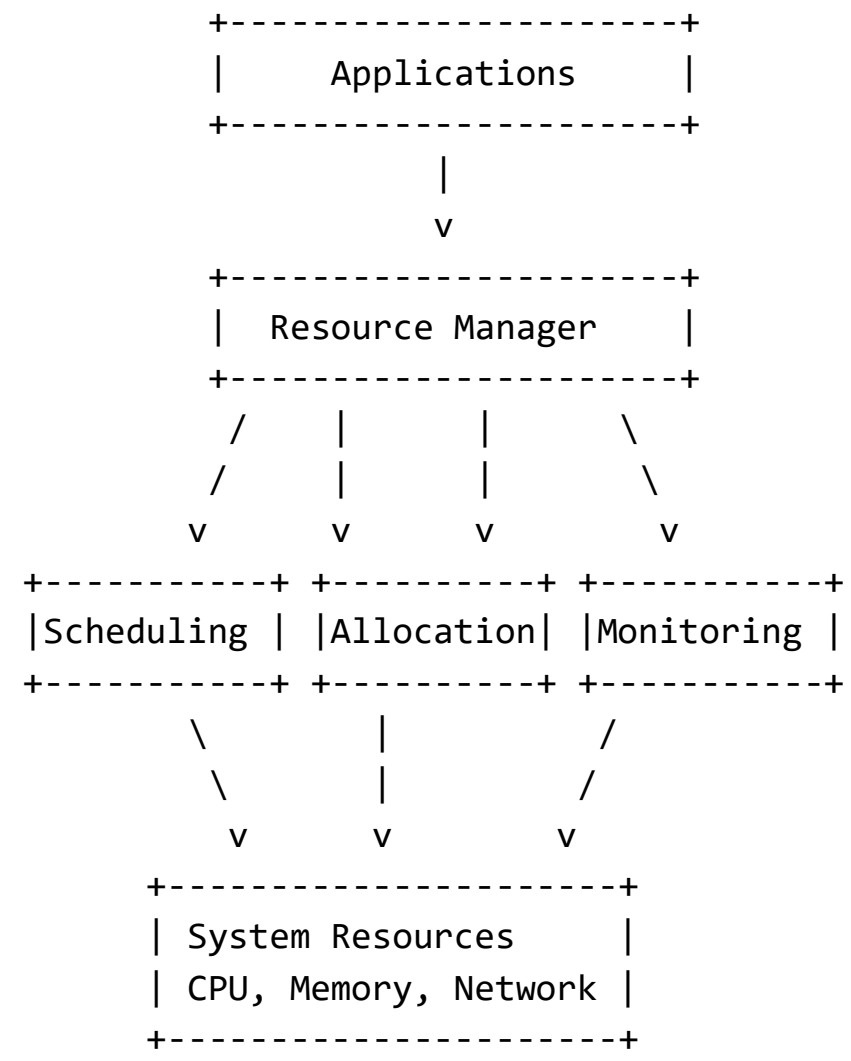
### 8. Fault Tolerance

Ensures resource availability despite failures.

#### Techniques

- Replication
- Backup systems
- Checkpointing

### Resource Management Architecture



## Resource Management in Distributed Multimedia Systems

In multimedia systems, resource management is very important because multimedia applications require:

- Real-time processing
- High bandwidth
- Synchronization
- QoS guarantees

## Multimedia Resource Requirements

| Resource | Requirement |
|---|---|
| CPU | Video/audio processing |
| Bandwidth | Streaming media |
| Buffer | Smooth playback |
| Storage | Large multimedia files |

## Functions in Multimedia Resource Management

### 1. QoS Management

Ensures:

- Low delay
- Controlled jitter
- Minimum packet loss

### 2. Bandwidth Allocation

Allocates sufficient network capacity for media streams.

### 3. Synchronization Management

Maintains audio-video synchronization.

### 4. Dynamic Adaptation

Adjusts quality according to available resources.

### Example

Reducing video quality during congestion.

## Resource Management Strategies

### 1. Centralized Resource Management

Single central manager controls all resources.

#### Advantages

- Simple management
- Easy monitoring

#### Disadvantages

- Single point of failure
- Poor scalability

### 2. Distributed Resource Management

Multiple managers control resources cooperatively.

#### Advantages

- Better scalability
- Fault tolerance

#### Disadvantages

- Complex coordination

### 3. Hierarchical Resource Management

Uses multiple levels of managers.

#### Example

Cluster managers under a global manager.

## Challenges in Resource Management

1. Heterogeneity
2. Dynamic workloads
3. Resource contention
4. Scalability
5. Fault handling
6. Security
7. QoS maintenance
8. Real-time constraints

## Advantages of Effective Resource Management

- Improved performance
- Better resource utilization
- Reduced congestion
- Increased reliability
- Fair resource sharing
- Better user experience

## Applications of Resource Management

- Cloud computing
- Distributed databases
- Multimedia systems
- Grid computing
- Operating systems
- Data centers

## Conclusion

Resource management is a fundamental function in distributed systems that ensures efficient allocation, scheduling, monitoring, and control of system resources. In distributed multimedia systems, effective resource management is essential for maintaining QoS, synchronization, real-time

performance, and efficient utilization of computing and communication resources.

- ♦ **Distributed Web Based Systems**
- ❖ **Architecture of Traditional Web-Based Systems**

## Architecture of Traditional Web-Based Systems

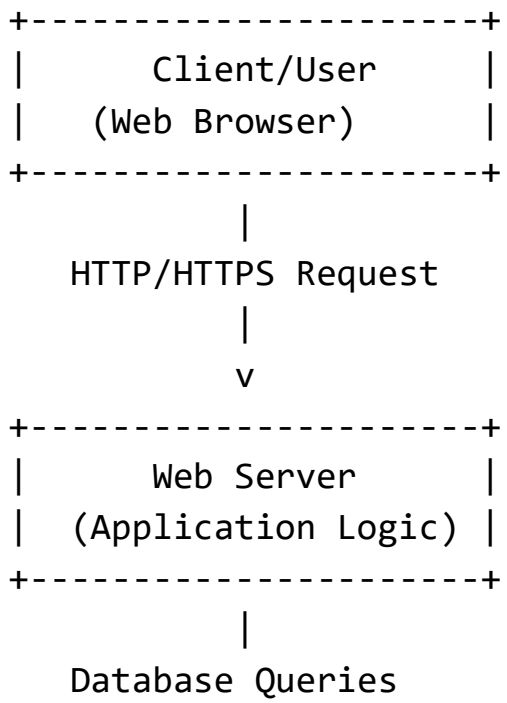
A **Traditional Web-Based System** follows a **client-server architecture** where users access web applications through web browsers over the internet or an intranet.

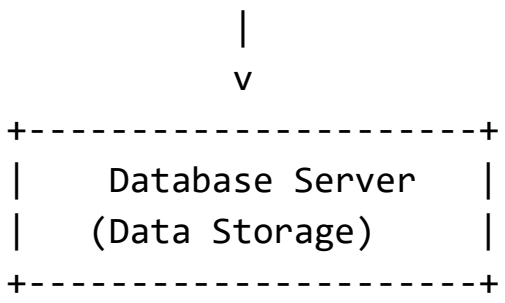
The architecture mainly consists of:

- Client tier
- Web/Application server tier
- Database tier

This architecture is commonly called a **3-tier architecture**.

## Diagram of Traditional Web-Based System Architecture






## Components of Traditional Web-Based Systems

### 1. Client Tier (Presentation Layer)

The client tier is the front-end interface through which users interact with the system.

#### Components

- Web browser
- User interface
- HTML pages
- CSS
- JavaScript

#### Functions

- Sends requests to server
- Displays web pages
- Accepts user input

#### Examples of Browsers

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

### 2. Web Server / Application Server Tier

This tier processes client requests and contains business logic.

### Responsibilities

- Handles HTTP requests
- Executes application logic
- Processes user authentication
- Generates dynamic web pages
- Communicates with database server

### Examples of Web Servers

- Apache HTTP Server
- Nginx
- Microsoft IIS

### Application Technologies

- PHP
- Java Servlets
- JSP
- ASP.NET
- Python
- Node.js

## 3. Database Tier (Data Layer)

The database server stores and manages application data.

### Responsibilities

- Data storage
- Query processing
- Transaction management
- Data retrieval

### Examples

- MySQL
- Oracle Database
- Microsoft SQL Server

## Working of Traditional Web-Based Systems

### Step 1: User Request

The user enters a URL in the browser.

Example:

https://example.com

The browser sends an HTTP/HTTPS request to the web server.

### Step 2: Request Processing

The web server:

- Receives the request
- Executes business logic
- Validates user input

### Step 3: Database Access

If data is needed:

- Server sends queries to database server.
- Database processes the request.

### Step 4: Response Generation

The server generates:

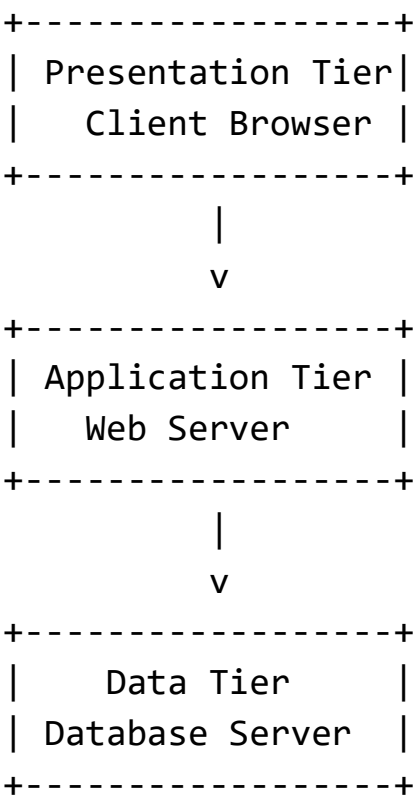
- HTML page
- JSON/XML data
- Dynamic content

### Step 5: Response to Client

The web server sends the response back to the browser.

The browser displays the webpage to the user.

### Three-Tier Architecture Representation




### Characteristics of Traditional Web-Based Systems

1. Centralized server architecture
2. Thin clients (browser-based access)
3. Request-response communication model
4. Shared database system
5. Platform independence through browsers

### Advantages of Traditional Web-Based Systems

#### 1. Easy Accessibility

Users can access applications from anywhere using a browser.

#### 2. Centralized Management

Applications and data are managed centrally on servers.

### 3. Platform Independence

Works on multiple operating systems and devices.

### 4. Easy Maintenance

Updates are applied on the server side only.

### 5. Cost Effective

No need for complex client-side installations.

## Limitations of Traditional Web-Based Systems

### 1. Server Overload

Heavy traffic may overload the server.

### 2. Single Point of Failure

Server failure may stop the entire system.

### 3. Network Dependency

Requires stable internet/network connection.

### 4. Scalability Issues

Traditional architectures may struggle with very large user loads.

### 5. Performance Bottlenecks

Frequent database access can reduce performance.

## Applications of Traditional Web-Based Systems

- E-commerce websites
- Online banking
- Educational portals
- Library systems
- Reservation systems
- Enterprise applications

## Conclusion

Traditional web-based systems use a client-server or three-tier architecture where clients interact with web servers that process requests and communicate with databases. This architecture provides centralized management, browser-based accessibility, and platform independence, making it widely used for web applications and online services.

- ❖ **Apache Web Server**

**May_Jun_2023**

Q6) b) What are web services? Describe with a suitable diagram the general organization of the Apache web server.

Ans. Web Services

## Definition

Web services are software components that enable communication and data exchange between different applications over a network using standard web protocols.

They allow heterogeneous applications developed in different:

- Programming languages
- Operating systems
- Platforms

to communicate and share data.

Web services mainly use:

- HTTP/HTTPS
- XML
- JSON
- SOAP
- REST

## Features of Web Services

1. Platform independent
2. Language independent
3. Loosely coupled
4. Reusable
5. Interoperable
6. Accessible over networks

## Types of Web Services

### 1. SOAP Web Services

SOAP (Simple Object Access Protocol) uses XML-based messaging.

#### Characteristics

- Protocol-based
- Highly secure
- Standardized communication

### 2. RESTful Web Services

REST (Representational State Transfer) uses HTTP methods.

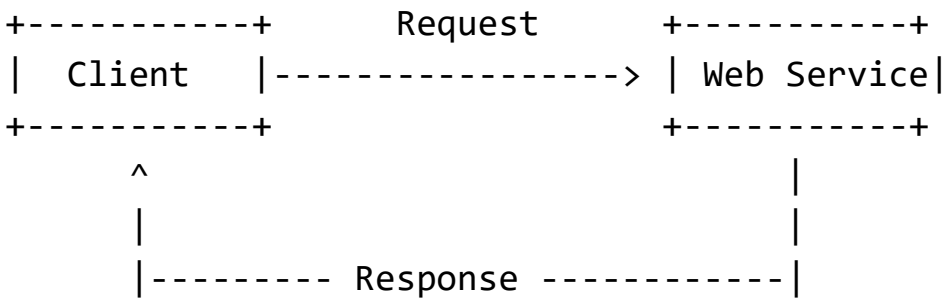
#### HTTP Methods

- GET
- POST
- PUT
- DELETE

#### Characteristics

- Lightweight
- Faster
- Uses JSON/XML

## Architecture of Web Services




## Applications of Web Services

- E-commerce systems
- Online banking
- Cloud computing
- Mobile applications
- Distributed systems

## Apache Web Server

The Apache HTTP Server is one of the most widely used web server software systems developed by the Apache Software Foundation.

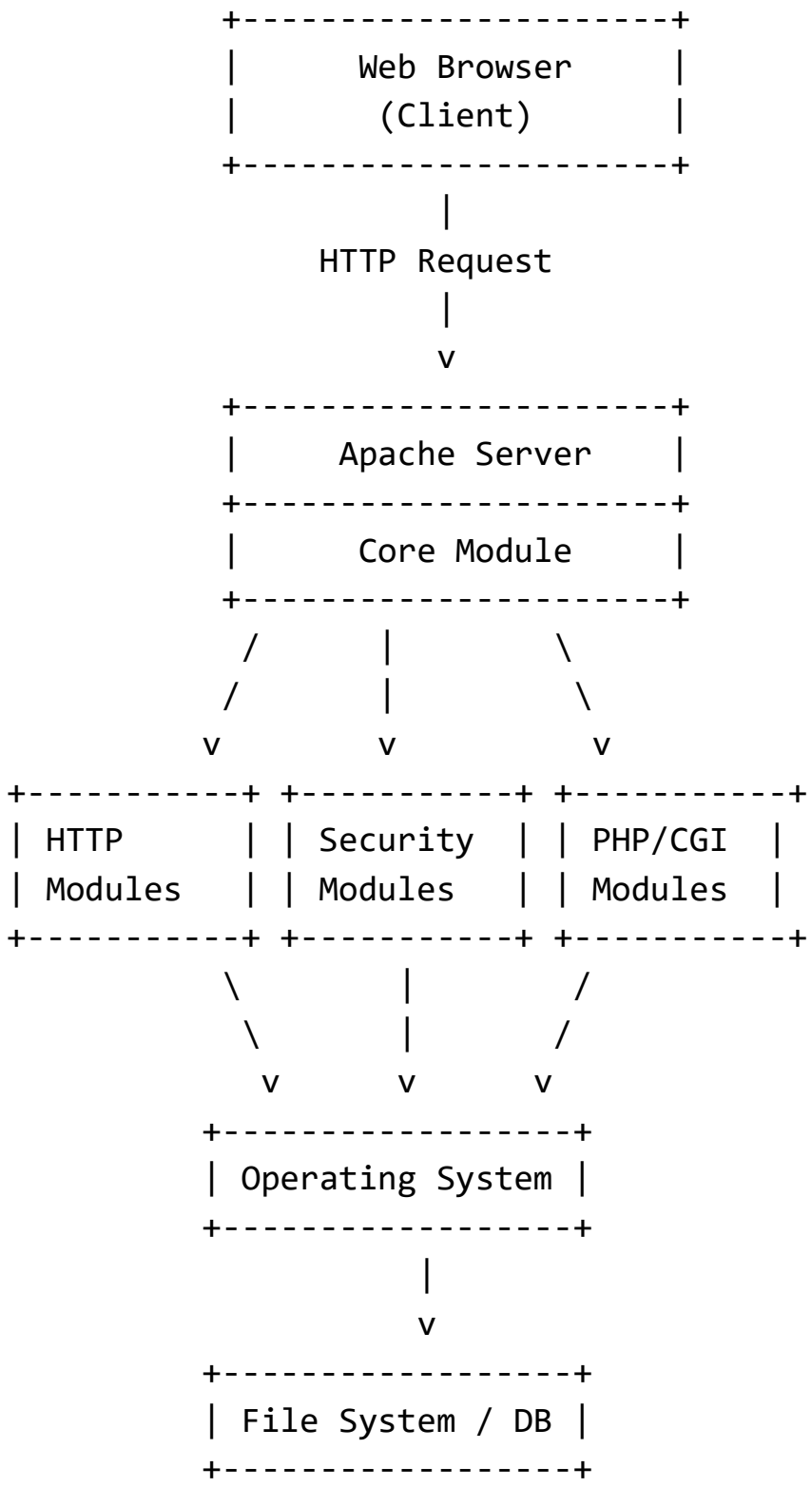
It is:

- Open source
- Cross-platform
- Highly configurable
- Modular in design

## General Organization of Apache Web Server

The Apache web server follows a modular architecture where the core server works together with multiple modules to process client requests.

## Diagram of Apache Web Server Organization



## Components of Apache Web Server

### 1. Client (Web Browser)

The client sends HTTP/HTTPS requests to the Apache server.

#### Examples

- Google Chrome
- Mozilla Firefox

### 2. Apache Core Module

The core module is the central part of the Apache server.

#### Responsibilities

- Request handling
- Configuration management
- Module management
- Process control

#### Functions

- Receives client requests
- Coordinates module execution
- Generates responses

### 3. HTTP Modules

Handle HTTP protocol operations.

#### Functions

- Process HTTP requests
- Manage headers
- Handle connections

#### Example Modules

- mod_http

- mod_headers

### 4. Security Modules

Provide authentication and security services.

#### Functions

- User authentication
- Access control
- SSL/TLS encryption

#### Example Modules

- mod_auth
- mod_ssl

### 5. PHP/CGI Modules

Support dynamic web content generation.

#### Functions

- Execute scripts
- Process server-side applications

#### Technologies

- PHP
- CGI
- Perl
- Python

#### Example Modules

- mod_php
- mod_cgi

### 6. Operating System Interface

Apache interacts with the operating system for:

- Process management
- Memory handling
- File access
- Networking

### 7. File System / Database

Stores:

- HTML files
- Images
- Scripts
- Databases

Apache retrieves requested resources from storage.

## Working of Apache Web Server

### Step 1: Client Request

Browser sends an HTTP request.

Example:

```
GET /index.html HTTP/1.1
```

### Step 2: Request Reception

Apache server receives the request.

### Step 3: Request Processing

The core server forwards the request to appropriate modules.

Examples:

- Security module checks permissions
- PHP module executes scripts

### Step 4: Resource Access

Apache accesses:

- Files
- Databases
- Scripts

through the operating system.

### Step 5: Response Generation

Server prepares the response:

- HTML page
- Dynamic content
- Error messages

### Step 6: Response to Client

Response is sent back to the browser.

## Features of Apache Web Server

1. Open source
2. Modular architecture
3. Cross-platform support
4. Virtual hosting
5. SSL/TLS support
6. Loadable modules
7. High reliability

## Advantages of Apache Web Server

- Free and open source
- Highly configurable
- Large community support
- Secure and stable

- Supports dynamic content

### Limitations of Apache Web Server

- Higher memory usage under heavy load
- Slower than some lightweight servers for static content
- Complex configuration for beginners

### Conclusion

Web services enable interoperable communication between distributed applications using standard web protocols. The Apache HTTP Server uses a modular architecture consisting of core modules, protocol handlers, security modules, and scripting modules to efficiently process web requests and provide web services to clients across networks.

❖ **Web Server Clusters**

**Nov_Dec_2023**

Q6) a) Describe, using the appropriate diagram, how a web service is implemented in horizontal distribution using web server clusters.

Ans. Horizontal Distribution of Web Services Using Web Server Clusters

### Introduction

In large-scale web applications, a single web server may not be sufficient to handle:

- Heavy user traffic
- Large numbers of requests
- High availability requirements

To solve this problem, web services are implemented using **horizontal distribution** through **web server clusters**.

## What is Horizontal Distribution?

Horizontal distribution means adding multiple servers that work together to provide the same service.

Instead of increasing the power of one server (**vertical scaling**), multiple servers are added to distribute workload.

This approach improves:

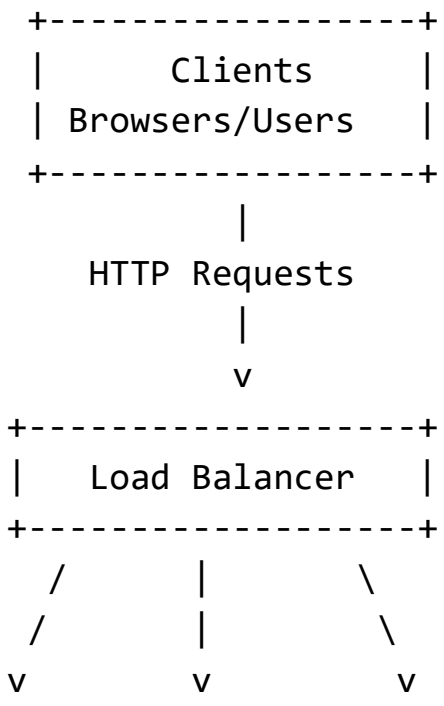
- Scalability
- Reliability
- Performance
- Availability

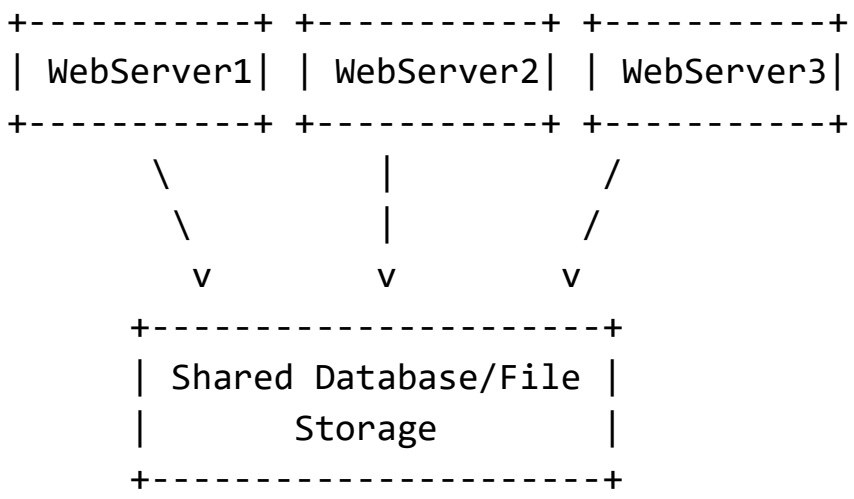
## Web Server Cluster

A **web server cluster** is a group of interconnected web servers that collectively process client requests.

Clients view the cluster as a single system.

## Architecture Diagram of Horizontal Distribution Using Web Server Clusters






## Components of the Architecture

### 1. Clients

Clients are users accessing the web service through:

- Browsers
- Mobile applications
- APIs

#### Functions

- Send HTTP/HTTPS requests
- Receive responses from servers

### 2. Load Balancer

The load balancer is the central distribution component.

#### Responsibilities

- Receives client requests
- Distributes requests among servers
- Prevents server overload
- Improves availability

### Load Balancing Algorithms

#### a) Round Robin

Requests distributed sequentially.

#### b) Least Connections

Requests sent to least loaded server.

#### c) IP Hashing

Distribution based on client IP.

### 3. Web Server Cluster

Multiple web servers process requests simultaneously.

#### Functions

- Execute web applications
- Process business logic
- Serve web pages
- Handle user sessions

#### Examples of Servers

- Apache HTTP Server
- Nginx

### 4. Shared Database/File Storage

All servers access common backend storage.

#### Stores

- Application data
- User information
- Session data
- Web files

### Advantages

- Data consistency
- Shared access

### Working of Horizontal Distribution

#### Step 1: Client Sends Request

A client sends an HTTP request to the web service.

#### Step 2: Load Balancer Receives Request

The load balancer acts as the entry point.

#### Step 3: Request Distribution

The load balancer forwards the request to one of the web servers.

Example:

```
Request 1 → WebServer1
Request 2 → WebServer2
Request 3 → WebServer3
```

#### Step 4: Request Processing

The selected server:

- Processes the request
- Accesses shared storage/database if needed

#### Step 5: Response Returned

The processed response is sent back to the client.

## Advantages of Horizontal Distribution

### 1. Scalability

Additional servers can easily be added to handle increased traffic.

### 2. High Availability

If one server fails:

- Other servers continue working.

### 3. Fault Tolerance

System continues operating despite hardware/software failures.

### 4. Better Performance

Workload is shared among multiple servers.

### 5. Load Balancing

Prevents any single server from becoming overloaded.

### 6. Easy Maintenance

Servers can be updated individually without stopping the entire system.

### Example Scenario

```
E-commerce Website:
10,000 users simultaneously access the website.
Load balancer distributes requests across multiple servers.
```

This prevents server crashes and improves response time.

## Challenges in Horizontal Distribution

### 1. Session Management

User sessions must remain consistent across servers.

### Solutions

- Shared session storage
- Sticky sessions

### 2. Data Consistency

All servers must access updated data.

### 3. Synchronization Complexity

Multiple servers require coordination.

### 4. Increased Infrastructure Cost

More servers and networking equipment are required.

## Types of Web Clustering

| Type | Description |
|---|---|
| Active-Active | All servers handle requests |
| Active-Passive | Backup server activates on failure |

## Comparison: Horizontal vs Vertical Scaling

| Feature | Horizontal Scaling | Vertical Scaling |
|---|---|---|
| Method | Add more servers | Increase server power |
| Scalability | High | Limited |
| Fault Tolerance | Better | Lower |
| Cost | Flexible | Expensive hardware |

### Applications of Web Server Clusters

- E-commerce websites
- Cloud platforms
- Social media systems
- Streaming services
- Banking systems
- Search engines

### Conclusion

Horizontal distribution using web server clusters is an effective method for implementing scalable and reliable web services. By using multiple web servers behind a load balancer, distributed systems can efficiently manage heavy traffic, improve performance, provide fault tolerance, and ensure high availability for modern web applications.

❖ **Communication by Hypertext Transfer Protocol**

## Communication by Hypertext Transfer Protocol (HTTP)

### Introduction

The Hypertext Transfer Protocol (HTTP) is the standard communication protocol used for transferring web pages and other resources between:

- Web clients (browsers)
- Web servers

HTTP is the foundation of communication on the World Wide Web (WWW).

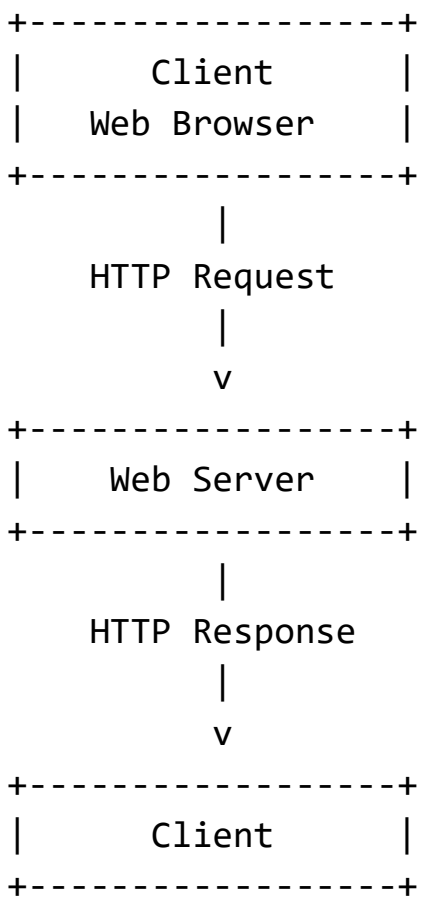
It operates at the **Application Layer** of the TCP/IP model.

### Basic HTTP Communication Model

HTTP follows a:

- Client-server architecture
- Request-response model

## HTTP Communication Diagram




## Components of HTTP Communication

### 1. Client

The client is usually a web browser or application.

#### Examples

- Google Chrome
- Mozilla Firefox
- Mobile apps

#### Functions

- Sends HTTP requests

- Receives server responses
- Displays web content

### 2. Web Server

The server stores and provides web resources.

#### Examples

- Apache HTTP Server
- Nginx

#### Functions

- Processes client requests
- Retrieves resources
- Sends responses

### 3. HTTP Protocol

Defines:

- Request format
- Response format
- Communication rules

## Working of HTTP Communication

### Step 1: URL Request

The user enters a URL in the browser.

Example:

https://www.example.com

### Step 2: DNS Resolution

The browser converts the domain name into an IP address using DNS.

### Step 3: TCP Connection

The client establishes a TCP connection with the server.

### Default Ports

- HTTP → Port 80
- HTTPS → Port 443

### Step 4: HTTP Request

The client sends an HTTP request message.

### Step 5: Server Processing

The web server:

- Processes the request
- Accesses files/databases
- Generates response

### Step 6: HTTP Response

The server sends an HTTP response back to the client.

### Step 7: Connection Close or Reuse

The connection may:

- Close
- Remain open for persistent communication

### Structure of HTTP Request Message

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Chrome
Accept: text/html
```

## Components of HTTP Request

| Component | Description |
|---|---|
| Request Line | Method, URL, version |
| Headers | Additional information |
| Body | Optional data |

## HTTP Request Methods

| Method | Purpose |
|---|---|
| GET | Retrieve resource |
| POST | Submit data |
| PUT | Update resource |
| DELETE | Remove resource |
| HEAD | Retrieve headers only |

## Structure of HTTP Response Message

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
<html>
...
</html>
```

## Components of HTTP Response

| Component | Description |
| --- | --- |
| Status Line | Version and status code |
| Headers | Response information |
| Body | Actual content |

## HTTP Status Codes

### 1xx – Informational

Example:

```
100 Continue
```

### 2xx – Success

Example:

```
200 OK
```

### 3xx – Redirection

Example:

```
301 Moved Permanently
```

### 4xx – Client Errors

Example:

```
404 Not Found
```

### 5xx – Server Errors

Example:

```
500 Internal Server Error
```

## Stateless Nature of HTTP

HTTP is a stateless protocol.

### Meaning

Each request is independent.

The server does not automatically remember previous requests.

## Maintaining State in HTTP

State is maintained using:

- Cookies
- Sessions
- Tokens

## HTTP vs HTTPS

| Feature | HTTP | HTTPS |
|---|---|---|
| Security | No encryption | Encrypted |
| Port | 80 | 443 |
| Protocol | Plain text | SSL/TLS secured |

## Persistent and Non-Persistent Connections

### Non-Persistent Connection

- One request per TCP connection
- Connection closes after response

### Persistent Connection

- Multiple requests use same connection
- Faster communication

Supported in HTTP/1.1.

## Features of HTTP

1. Simple protocol
2. Stateless communication
3. Media independent
4. Extensible through headers
5. Supports distributed systems

## Advantages of HTTP

- Simple implementation
- Widely supported
- Platform independent
- Supports multimedia transmission
- Flexible communication

## Limitations of HTTP

- Stateless nature
- Security issues in plain HTTP
- Additional overhead for repeated connections

## Applications of HTTP

- Web browsing
- REST APIs
- Web services
- Cloud applications
- Multimedia streaming

## Conclusion

The Hypertext Transfer Protocol enables communication between web clients and servers using a request-response model. It forms the backbone of web-based distributed systems and supports the transfer of web pages, multimedia content, and web services across the internet.

❖ **Synchronization**

## Synchronization in Distributed Systems

### Definition

Synchronization is the process of coordinating multiple processes, threads, or systems so that they execute operations in a correct and orderly manner while sharing resources or exchanging information.

In distributed systems, synchronization ensures:

- Correct execution order
- Consistent data access
- Coordination among processes
- Avoidance of conflicts

### Need for Synchronization

Synchronization is required because multiple processes may:

- Access shared resources simultaneously
- Communicate concurrently
- Execute independently on different machines

Without synchronization:

- Data inconsistency may occur
- Race conditions may happen
- Deadlocks can arise
- System reliability decreases

### Example of Synchronization Problem

```
Shared Variable = 100
Process P1 adds 50
Process P2 subtracts 30
Without synchronization:
Incorrect final value may occur.
```

## Objectives of Synchronization

1. Maintain data consistency
2. Avoid race conditions
3. Ensure mutual exclusion
4. Coordinate distributed processes
5. Maintain correct execution order
6. Support reliable communication

## Types of Synchronization

### 1. Clock Synchronization

Ensures all computers in a distributed system maintain approximately the same time.

#### Importance

- Event ordering
- Logging
- Distributed transactions

#### Methods

- Physical clock synchronization
- Logical clock synchronization

### Physical Clock Synchronization

Synchronizes actual system clocks.

#### Algorithms

- Cristian's Algorithm
- Berkeley Algorithm
- NTP (Network Time Protocol)

### Logical Clock Synchronization

Orders events logically without depending on physical time.

### Algorithms

- Lamport Logical Clock
- Vector Clock

## 2. Process Synchronization

Coordinates execution of concurrent processes.

### Goals

- Prevent simultaneous conflicting access
- Maintain execution correctness

### Mutual Exclusion

Only one process can access a critical resource at a time.

### Critical Section

Part of a program where shared resources are accessed.

```
Process
   Entry Section
   Critical Section
   Exit Section
   Remainder Section
```

### Synchronization Mechanisms

#### 1. Locks

A lock prevents multiple processes from accessing shared resources simultaneously.

#### Types

- Binary lock
- Read-write lock

### 2. Semaphores

Special synchronization variables used to control access.

#### Operations

- wait()
- signal()

### 3. Monitors

High-level synchronization construct containing:

- Shared data
- Procedures
- Synchronization logic

### 4. Message Passing

Processes synchronize by exchanging messages.

#### Operations

- send()
- receive()

Used extensively in distributed systems.

### Distributed Synchronization

In distributed systems:

- Processes run on different machines
- No shared memory exists
- Communication occurs over networks

Synchronization becomes more complex because of:

- Network delay
- Clock differences
- Node failures

## Distributed Mutual Exclusion

Ensures only one distributed process accesses a shared resource at a time.

## Approaches to Distributed Mutual Exclusion

### 1. Centralized Algorithm

Single coordinator grants permission.

#### Advantages

- Simple implementation

#### Disadvantages

- Single point of failure

### 2. Token-Based Algorithm

A token grants permission to enter critical section.

#### Example

Token Ring Algorithm

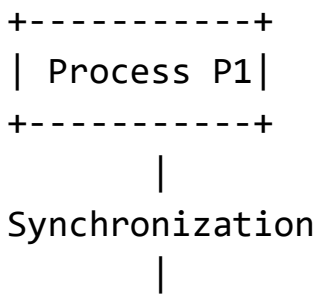
### 3. Permission-Based Algorithm

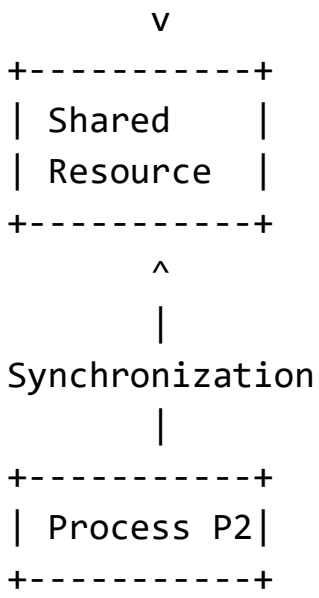
Processes exchange permission messages.

#### Example

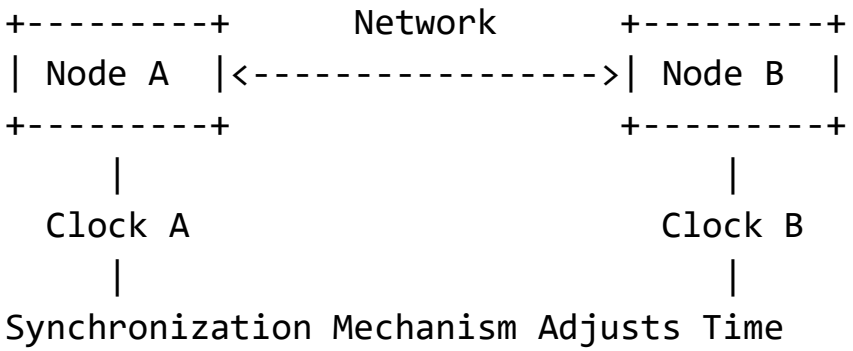
Ricart-Agrawala Algorithm

## Synchronization Diagram






## Clock Synchronization Diagram




## Synchronization in Multimedia Systems

Multimedia systems require synchronization for:

- Audio-video coordination
- Real-time streaming
- Interactive communication

## Types of Multimedia Synchronization

### 1. Intra-media Synchronization

Synchronization within the same media stream.

#### Example

Correct sequence of video frames.

### 2. Inter-media Synchronization

Synchronization between different media streams.

#### Example

Lip synchronization between audio and video.

## Challenges in Synchronization

1. Network latency
2. Clock drift
3. Process concurrency
4. Distributed failures
5. Scalability
6. Deadlocks
7. Message delays

## Advantages of Synchronization

- Data consistency
- Correct execution
- Reliable communication
- Efficient resource sharing
- Prevention of race conditions

## Disadvantages

- Increased overhead

- Communication delays
- Complexity in distributed systems
- Potential deadlocks

### Applications of Synchronization

- Distributed databases
- Operating systems
- Banking systems
- Multimedia systems
- Cloud computing
- Real-time systems

### Conclusion

Synchronization is a fundamental concept in distributed systems that coordinates concurrent processes and ensures correct access to shared resources. It plays a critical role in maintaining consistency, reliability, and proper execution order in distributed computing and multimedia applications.

❖ **Web Proxy Caching**

## Web Proxy Caching

### Introduction

Web Proxy Caching is a technique used to improve web performance by storing copies of frequently accessed web pages and web objects closer to users.

A **proxy cache server** acts as an intermediary between:

- Clients (web browsers)
- Web servers

When users request web content:

- The proxy checks whether the content is already stored in its cache.
- If available, it serves the content directly.
- Otherwise, it retrieves the content from the original server and stores a copy for future requests.

## Definition

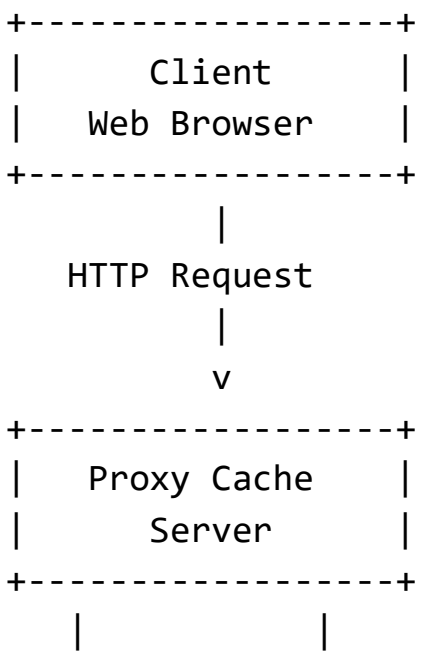
A **Web Proxy Cache** is a server that stores copies of web resources such as:

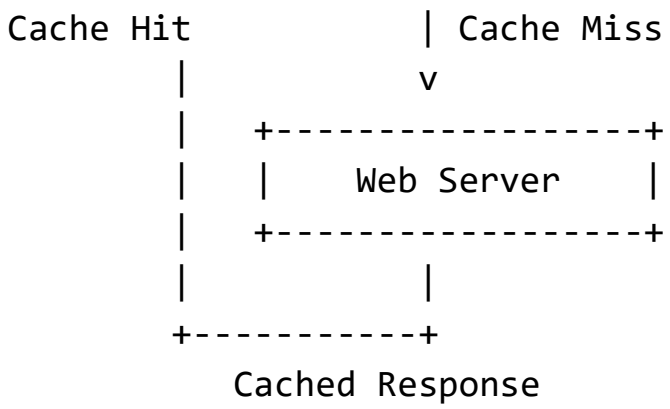
- HTML pages
- Images
- Videos
- Scripts
- Documents

to reduce:

- Network traffic
- Response time
- Server load

## Architecture of Web Proxy Caching






## Working of Web Proxy Caching

### Step 1: Client Request

The client requests a web page or object.

Example:

```
GET /index.html HTTP/1.1
```

### Step 2: Request to Proxy Server

The request is first sent to the proxy cache server.

### Step 3: Cache Lookup

The proxy checks whether the requested object exists in cache.

### Case 1: Cache Hit

If the object is available:

- Proxy directly sends cached content to client.

### Result

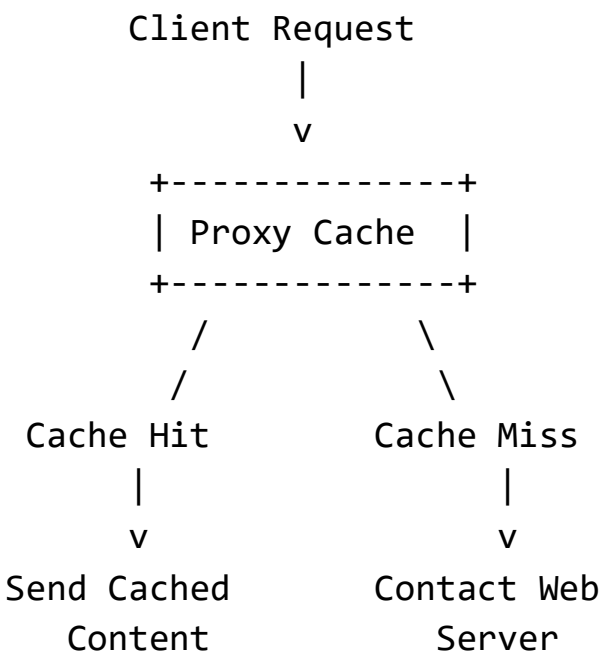
- Faster response
- Reduced internet traffic

### Case 2: Cache Miss

If object is not found:

- Proxy contacts original web server.
- Retrieves content.
- Stores copy in cache.
- Sends response to client.

### Cache Hit and Cache Miss Diagram




### Components of Web Proxy Caching

#### 1. Client

Requests web resources.

Examples:

- Google Chrome
- Mozilla Firefox

### 2. Proxy Cache Server

Stores cached web objects.

#### Functions

- Request filtering
- Cache management
- Content delivery
- Traffic reduction

### 3. Origin Web Server

Original source of web content.

## Types of Proxy Caching

### 1. Forward Proxy Cache

Located between client and internet.

#### Purpose

- Speeds up client access
- Filters requests

### 2. Reverse Proxy Cache

Located in front of web servers.

#### Purpose

- Reduces backend server load
- Improves scalability

#### Forward Proxy Diagram

```
Client --> Proxy Cache --> Internet/Web Server
```

#### Reverse Proxy Diagram

```
Client --> Reverse Proxy --> Web Servers
```

## Cache Replacement Policies

When cache becomes full, old objects must be removed.

### 1. FIFO (First In First Out)

Oldest object removed first.

### 2. LRU (Least Recently Used)

Least recently accessed object removed.

### 3. LFU (Least Frequently Used)

Least frequently used object removed.

### 4. Random Replacement

Random object removed.

## Cache Consistency

Cached content may become outdated.

### Solutions

- Expiration time
- Validation checks
- Conditional requests

### Example

```
If webpage updated on server,
proxy must refresh cached copy.
```

## Advantages of Web Proxy Caching

### 1. Reduced Response Time

Users receive content faster from nearby cache.

### 2. Reduced Bandwidth Usage

Repeated downloads from internet are avoided.

### 3. Reduced Server Load

Fewer requests reach original server.

### 4. Improved Scalability

Supports large numbers of users efficiently.

### 5. Better Network Performance

Less congestion on network links.

### 6. Increased Availability

Cached content may still be available if origin server fails temporarily.

## Limitations of Web Proxy Caching

### 1. Stale Data Problem

Cached pages may become outdated.

### 2. Cache Management Complexity

Requires efficient replacement and consistency mechanisms.

### 3. Dynamic Content Challenges

Frequently changing content is difficult to cache.

### 4. Security and Privacy Issues

Sensitive data should not be cached improperly.

## Applications of Web Proxy Caching

- Web browsers

- Content Delivery Networks (CDNs)
- Corporate networks
- Educational institutions
- Internet Service Providers (ISPs)

### Examples of Proxy Cache Software

- Squid
- Varnish Cache
- Nginx

### Difference Between Proxy Server and Proxy Cache

| Feature | Proxy Server | Proxy Cache |
|---|---|---|
| Main Purpose | Intermediary communication | Store cached content |
| Performance Improvement | Limited | Significant |
| Storage | May not store data | Stores web objects |

### Conclusion

Web proxy caching improves the efficiency and scalability of web-based distributed systems by storing frequently accessed web resources closer to users. It reduces bandwidth consumption, lowers server load, and provides faster response times, making it an essential technique in modern web infrastructure and internet communication systems.

**Nov_Dec_2023**

Q6) b) Explain in brief, the two places of client-side web caching? Explain cooperative caching with suitable diagram.

Ans. Client-Side Web Caching

### Introduction

Client-side web caching stores web content closer to the user to improve:

- Web access speed
- Network efficiency
- User experience

Frequently accessed web objects such as:

- HTML pages
- Images
- CSS files
- JavaScript files

are temporarily stored so repeated downloads are avoided.

### Two Places of Client-Side Web Caching

Client-side caching mainly occurs at two places:

1. Browser Cache
2. Proxy Cache

### 1. Browser Cache

#### Definition

A browser cache stores web resources locally on the client machine inside the web browser.

#### Stored Objects

- HTML pages
- Images
- Videos
- CSS
- JavaScript

#### Location

Stored on:

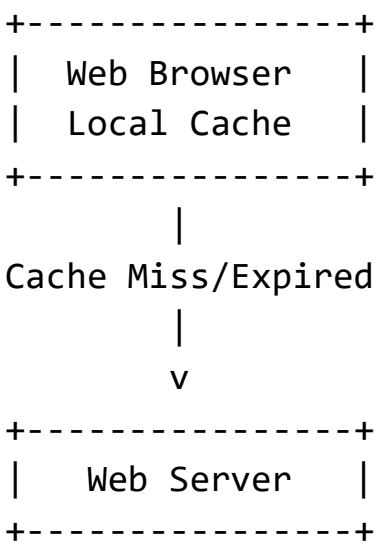
- Client memory
- Local disk

## Browser Cache Working

When a user revisits a webpage:

- Browser checks local cache first.
- If object exists and is valid:
  - Content is loaded locally.
- Otherwise:
  - Browser requests object from server.

## Browser Cache Diagram




## Advantages of Browser Cache

- Faster webpage loading
- Reduced bandwidth usage
- Lower server load
- Better user experience

## Limitations

- Limited storage space
- Stale/outdated content
- Cache consistency problems

## 2. Proxy Cache

### Definition

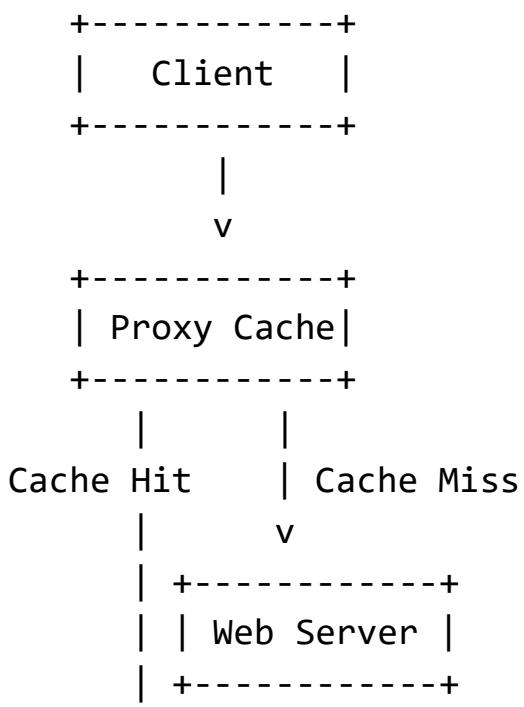
A proxy cache is a shared cache server located between clients and web servers.

It stores copies of web objects for multiple users.

### Proxy Cache Working

- Client sends request to proxy server.
- Proxy checks its cache.
- If object exists:
  - Sends cached copy.
- Else:
  - Retrieves from web server
  - Stores object for future use

### Proxy Cache Diagram



+------+

## Advantages of Proxy Cache

- Shared among many users
- Reduces internet traffic
- Improves organizational network performance

## Difference Between Browser Cache and Proxy Cache

| Feature | Browser Cache | Proxy Cache |
|---|---|---|
| Location | Client machine | Intermediate server |
| Accessibility | Single user | Multiple users |
| Storage Scope | Personal | Shared |
| Speed | Very fast | Fast |

## Cooperative Caching

### Definition

Cooperative caching is a caching technique where multiple cache servers cooperate and share cached web objects with each other.

Instead of fetching data only from the original web server:

- A cache may obtain data from neighboring caches.

This improves:

- Cache hit ratio
- Response time
- Bandwidth efficiency

## Need for Cooperative Caching

Single cache servers may:

- Have limited storage

- Miss requested objects frequently

Cooperative caching solves this by:

- Sharing cached data among distributed caches.

## Working of Cooperative Caching

### Step 1: Client Request

Client requests a web object.

### Step 2: Local Cache Check

Local cache checks whether object exists.

### Step 3: Neighbor Cache Query

If object not found:

- Local cache asks neighboring cooperative caches.

### Step 4: Object Retrieval

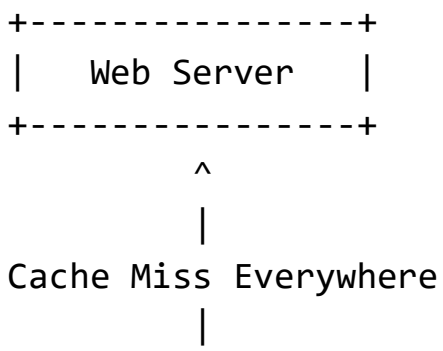
#### Case 1: Neighbor Cache Hit

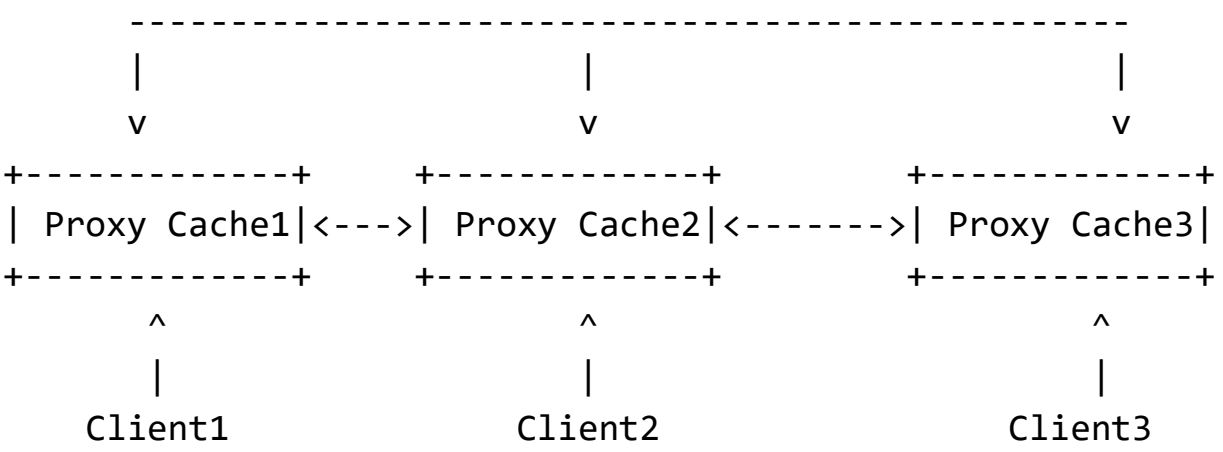
Object obtained from neighboring cache.

#### Case 2: Total Cache Miss

Object fetched from original web server.

## Cooperative Caching Diagram






### Example of Cooperative Caching

```
Client1 requests video file.
Proxy Cache1 does not have file.
Cache1 checks Cache2.
Cache2 contains file.
Cache2 sends file to Cache1.
```

### Advantages of Cooperative Caching

#### 1. Higher Cache Hit Ratio

Objects may be found in neighboring caches.

#### 2. Reduced Internet Traffic

Less communication with distant web servers.

#### 3. Faster Response Time

Nearby caches respond faster.

#### 4. Better Bandwidth Utilization

Reduces redundant downloads.

### 5. Improved Scalability

Supports large distributed web systems efficiently.

## Challenges of Cooperative Caching

### 1. Cache Coordination Complexity

Caches must communicate and coordinate efficiently.

### 2. Cache Consistency

Maintaining updated copies across caches is difficult.

### 3. Additional Communication Overhead

Caches exchange messages frequently.

### 4. Security Concerns

Shared cache access may introduce security risks.

## Applications of Cooperative Caching

- Content Delivery Networks (CDNs)
- Distributed web systems
- Educational networks
- ISP proxy systems
- Large enterprise networks

## Conclusion

Client-side web caching improves web performance through browser caching and proxy caching. Cooperative caching further enhances efficiency by allowing multiple cache servers to share cached content, resulting in improved cache hit rates, reduced bandwidth consumption, faster web access, and better scalability in distributed web systems.

- **Case Study**
- **The Global Name Service**

## Case Study: The Global Name Service (GNS)

### Introduction

The **Global Name Service (GNS)** is a distributed naming service designed for large-scale distributed systems and internet-based applications.

Its main purpose is to:

- Map names to resources
- Support scalability
- Provide location transparency
- Enable efficient resource discovery

GNS is similar in concept to the Domain Name System (DNS), but is designed to support more general distributed object and service naming.

### Need for Global Name Service

In distributed systems:

- Resources are distributed across multiple machines.
- Users access resources using logical names rather than physical addresses.

Without a naming service:

- Resource location becomes difficult.
- Scalability problems occur.
- System management becomes complex.

### Objectives of GNS

1. Provide global naming
2. Ensure scalability
3. Support distributed environments
4. Enable location transparency

5. Provide fault tolerance
6. Support replication and caching
7. Ensure efficient name resolution

## Basic Concept of GNS

The Global Name Service maintains mappings between:

- Human-readable names

and

- Resource identifiers or addresses

## Example

```
service.company.com
        ↓
192.168.1.10
```

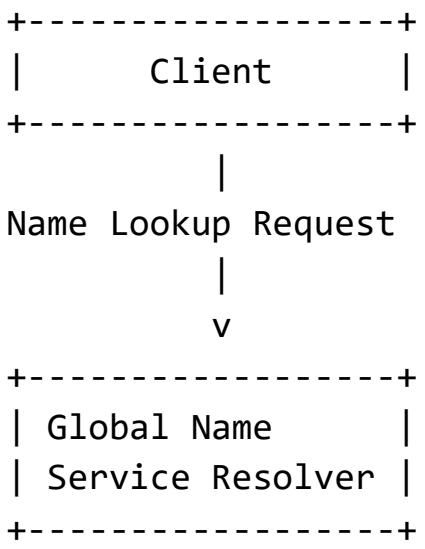
or

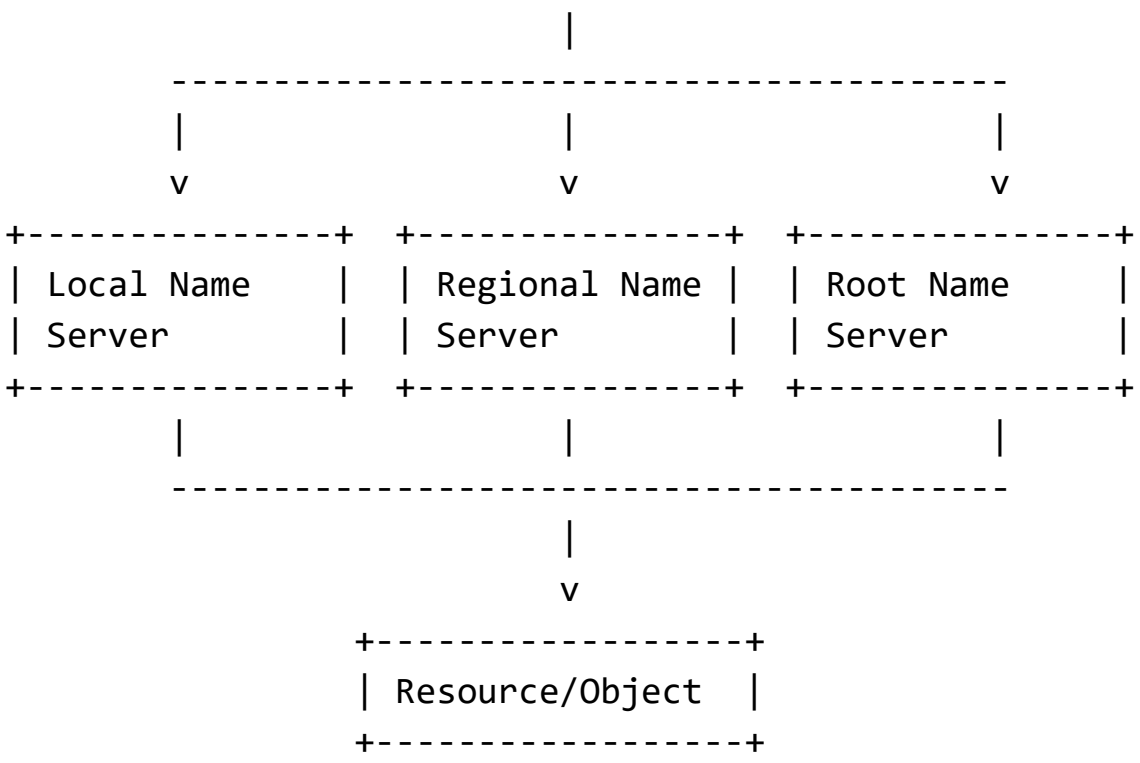
```
UserName → Distributed Object Location
```

## Architecture of Global Name Service

GNS generally follows a hierarchical and distributed architecture.

## GNS Architecture Diagram






## Components of GNS

### 1. Client

The client requests resource names or object locations.

### Functions

- Sends name lookup requests
- Receives resolved addresses

### 2. Name Resolver

Responsible for resolving names into addresses or resource references.

### Functions

- Searches naming hierarchy
- Contacts name servers
- Returns results to client

### 3. Name Servers

Store naming information and mappings.

#### Types

#### a) Root Name Server

Top-level naming authority.

#### b) Regional Name Server

Handles regional/domain information.

#### c) Local Name Server

Stores local resource mappings.

### 4. Resource/Object

The actual distributed resource:

- File
- Service
- Web server
- Distributed object

## Working of Global Name Service

### Step 1: Client Request

Client sends a name resolution request.

Example:

```
Find address of service.example.com
```

### Step 2: Resolver Processing

Resolver checks:

- Local cache
- Local name servers

### Step 3: Hierarchical Search

If name not found:

- Request moves upward through hierarchy.

### Step 4: Name Resolution

Appropriate name server returns:

- IP address
- Object reference
- Service location

### Step 5: Response to Client

Resolved information is returned to the client.

## Features of Global Name Service

### 1. Scalability

Supports very large distributed systems.

### 2. Distribution

Naming data distributed across multiple servers.

### 3. Replication

Multiple copies improve:

- Availability
- Reliability

### 4. Caching

Frequently used mappings stored locally for faster access.

### 5. Fault Tolerance

Failure of one server does not stop the entire system.

### 6. Transparency

Users access resources through logical names.

## Naming Structure in GNS

GNS usually uses hierarchical naming.

Example:

```
root
 └── organization
      └── department
           └── service
```

## Advantages of GNS

### 1. Easy Resource Access

Users need not remember physical addresses.

### 2. Location Transparency

Resources may move without changing names.

### 3. Improved Scalability

Distributed hierarchy supports large systems.

### 4. Better Performance

Caching reduces lookup time.

### 5. Reliability

Replication improves fault tolerance.

## Challenges in GNS

### 1. Consistency Maintenance

Replicated naming data must remain synchronized.

### 2. Lookup Latency

Global searches may increase delay.

### 3. Security Issues

Unauthorized modifications must be prevented.

### 4. Scalability Complexity

Large distributed systems require complex management.

## Applications of Global Name Service

- Internet DNS systems
- Cloud computing
- Distributed file systems
- Web services
- Peer-to-peer systems
- Grid computing

## Comparison Between DNS and GNS

| Feature | DNS | GNS |
|---|---|---|
| Main Purpose | Domain-to-IP mapping | General resource naming |
| Scope | Internet domains | Distributed objects/resources |
| Flexibility | Limited | More flexible |
| Resource Support | Mostly hosts | Multiple resource types |

## Conclusion

The Global Name Service is an essential component of distributed systems that provides scalable and transparent naming and resource discovery mechanisms. By using distributed hierarchical name servers, caching, and replication, GNS enables efficient access to distributed resources while supporting fault tolerance, scalability, and location transparency in large-scale distributed environments.

- ❖ **The X.500 Directory Service**

**May_Jun_2023**

Q5) b) What is a Directory Service? What is the difference between DNS and x500? Describe in detail the components of X.500 service architecture.

Ans. Directory Service

## Definition

A **Directory Service** is a distributed database system that stores, organizes, and provides access to information about:

- Users
- Computers
- Network resources
- Services
- Organizations

It allows users and applications to:

- Locate resources
- Retrieve information
- Manage distributed system objects efficiently

Directory services are widely used in:

- Distributed systems
- Enterprise networks
- Internet services

- Authentication systems

## Functions of a Directory Service

1. Naming resources
2. Resource discovery
3. User authentication
4. Access control
5. Information lookup
6. Distributed resource management

## Example of Directory Information

```
User Name: John
Email: john@example.com
Department: Sales
Phone: 9876543210
```

## Characteristics of Directory Services

- Hierarchical organization
- Distributed operation
- Scalability
- Replication support
- Efficient searching
- Security mechanisms

## DNS vs X.500

The Domain Name System (DNS) and X.500 are both naming and directory services, but they differ significantly in purpose and functionality.

## Difference Between DNS and X.500

| Feature | DNS | X.500 |
|---|---|---|
| Full Form | Domain Name System | X.500 Directory Service |
| Main Purpose | Maps domain names to IP addresses | Stores general directory information |
| Data Type | Limited resource records | Rich object information |
| Structure | Hierarchical domain tree | Hierarchical Directory Information Tree |
| Information Stored | Host/domain addresses | Users, organizations, services, devices |
| Query Complexity | Simple lookups | Complex searches |
| Protocol Used | DNS Protocol | DAP (Directory Access Protocol) |
| Flexibility | Less flexible | Highly flexible |
| Scalability | Internet-scale naming | Enterprise/global directory services |
| Security | Limited traditional support | Strong authentication mechanisms |
| Example | www.example.com ↗ → IP | Employee directory information |

## X.500 Directory Service

### Introduction

X.500 is a set of standards developed by:

- ITU-T
- ISO

for distributed directory services.

It defines:

- Directory architecture
- Naming conventions
- Communication protocols
- Data organization

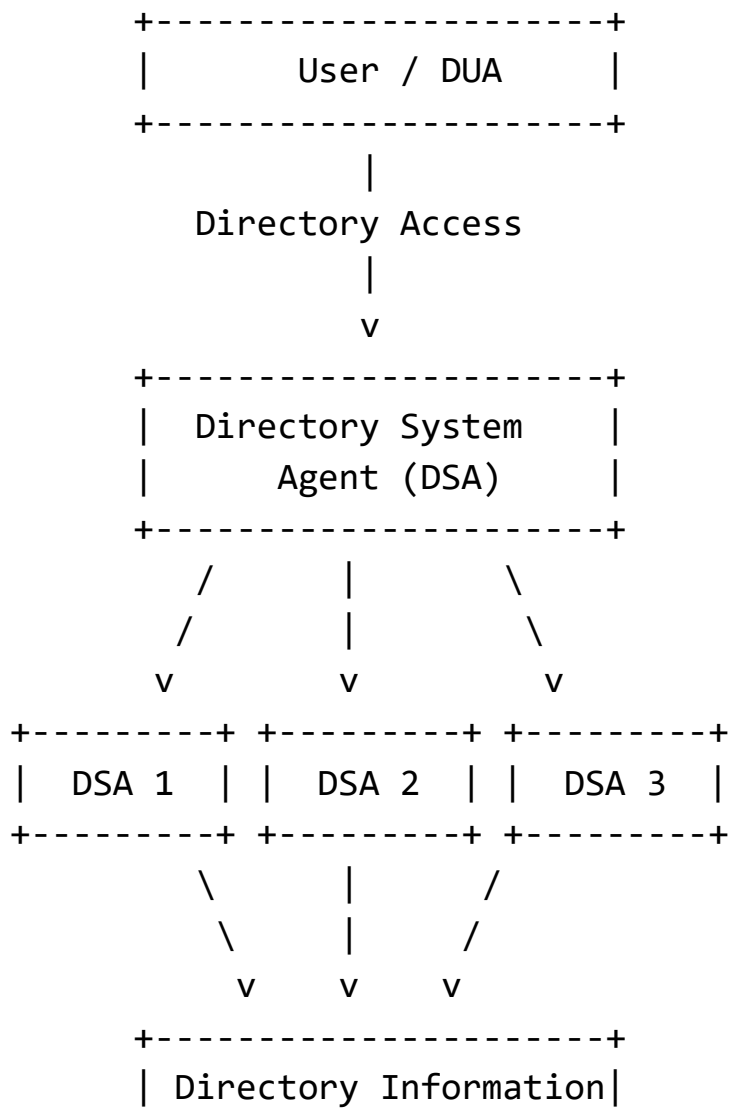
## Objectives of X.500

1. Global directory service
2. Distributed information management
3. Standardized naming
4. Efficient searching
5. Interoperability among systems

## X.500 Service Architecture

The X.500 architecture is based on a distributed hierarchical model.

## X.500 Architecture Diagram



```
|        Base (DIB)     |
+-----------------------+
```

## Components of X.500 Service Architecture

### 1. Directory User Agent (DUA)

#### Definition

The DUA is the client-side component used by users/applications to access directory services.

#### Functions

- Sends directory queries
- Receives search results
- Provides user interface

#### Example Operations

- Search employee information
- Retrieve email addresses

### 2. Directory System Agent (DSA)

#### Definition

The DSA is the server-side component that manages directory information.

#### Functions

- Stores directory entries
- Processes client requests
- Communicates with other DSAs
- Maintains distributed directory database

#### Responsibilities

- Query processing
- Authentication

- Replication management

### 3. Directory Information Base (DIB)

#### Definition

The DIB is the distributed database storing directory entries.

#### Contains

- User information
- Organization details
- Services
- Network resources

### 4. Directory Information Tree (DIT)

#### Definition

The DIT is the hierarchical structure used to organize directory entries.

#### Similar To

A tree-like file system hierarchy.

#### Example of DIT

```
Root
 └── Country
      └── Organization
           └── Department
                └── User
```

### 5. Directory Access Protocol (DAP)

#### Definition

DAP is the communication protocol between:

- DUA
- DSA

### Functions

- Searching
- Reading entries
- Updating information

## 6. Directory Operational Protocols

Used for communication among DSAs.

### Important Protocols

| Protocol | Function |
|---|---|
| DSP | Directory System Protocol |
| DISP | Directory Information Shadowing Protocol |
| DOP | Directory Operational Binding Management Protocol |

### X.500 Naming Structure

X.500 uses distinguished names (DNs).

### Example

```
CN=John Smith,
OU=Sales,
O=ABC Company,
C=IN
```

Where:

- CN → Common Name
- OU → Organizational Unit
- O → Organization
- C → Country

### Operations Supported by X.500

1. Read

2. Search
3. Add entry
4. Delete entry
5. Modify entry
6. Compare entries

### Features of X.500

- Hierarchical naming
- Distributed architecture
- Replication support
- Strong authentication
- Standardized protocols
- Complex query capability

### Advantages of X.500

#### 1. Global Directory Support

Can support worldwide distributed directories.

#### 2. Rich Information Storage

Stores detailed object information.

#### 3. Scalability

Hierarchical structure supports growth.

#### 4. Strong Security

Supports authentication and access control.

#### 5. Distributed Management

Information distributed across multiple DSAs.

## Limitations of X.500

### 1. Complexity

Architecture and protocols are complex.

### 2. High Overhead

DAP communication can be resource intensive.

### 3. Difficult Implementation

More difficult compared to DNS.

## Applications of X.500

- Enterprise directories
- Email directories
- Telecommunication systems
- Distributed authentication systems
- Organizational information systems

## Relationship with LDAP

Lightweight Directory Access Protocol (LDAP) is a simplified version of X.500 directory access mechanisms.

LDAP became more popular because:

- Simpler
- Lightweight
- Internet friendly

## Conclusion

A directory service provides organized and distributed access to information about users, services, and network resources. X.500 is a comprehensive directory service architecture that uses hierarchical naming, distributed

agents, and standardized protocols to support scalable and secure directory management, while DNS mainly focuses on domain name resolution.

- ❖ **BitTorrent**

**May_Jun_2023**

Q6) a) Describe the design of a peer-to-peer download system designed to support very large multimedia files. How does the BitTorrent protocol operate?

**May_Jun_2025**

Q6) a) Describe the design of a peer-to-peer file sharing application designed to support very large multimedia files. How does the BitTorrent protocol operate?

Ans. Peer-to-Peer (P2P) Download System for Large Multimedia Files

## Introduction

A **Peer-to-Peer (P2P)** system is a distributed architecture in which computers (called peers) share resources directly with each other without relying completely on a central server.

P2P systems are widely used for:

- File sharing
- Multimedia distribution
- Large data transfer
- Content distribution

They are especially useful for distributing very large multimedia files such as:

- Movies
- Music collections
- Software images
- High-definition videos

One of the most popular P2P file-sharing protocols is BitTorrent.

## Need for P2P Multimedia Sharing

Traditional client-server systems face problems when distributing large multimedia files:

- Server overload
- High bandwidth consumption
- Scalability limitations
- Single point of failure

P2P systems solve these problems by allowing peers to both:

- Download data
- Upload data

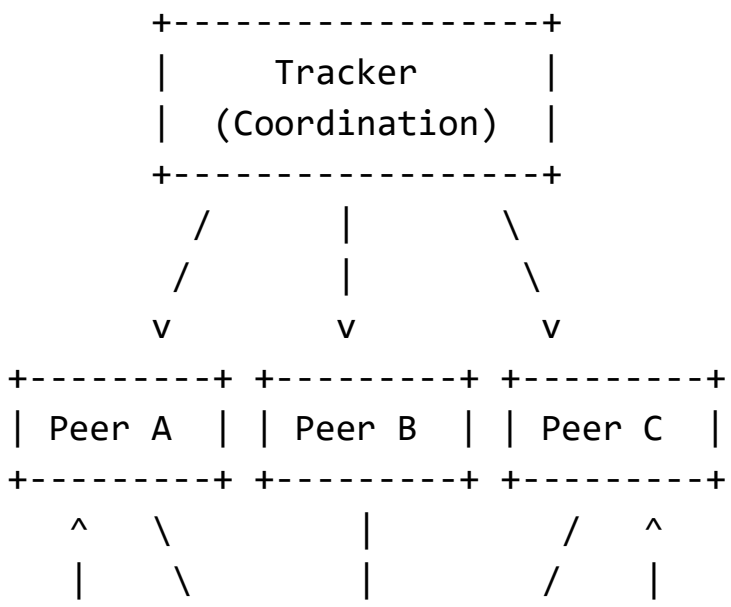
simultaneously.

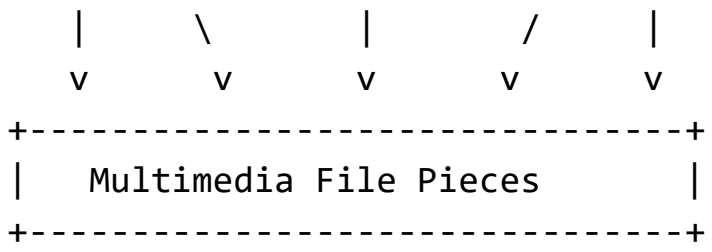
## Design of a P2P Download System

In a P2P multimedia sharing system:

- Each node acts as both client and server.
- Files are divided into smaller pieces.
- Peers exchange pieces directly.

## P2P File Sharing Architecture Diagram






## Components of a P2P Multimedia Sharing System

### 1. Peers

Peers are participating computers in the network.

#### Functions

- Download file pieces
- Upload file pieces
- Store shared data

Each peer can act as:

- Client
- Server

### 2. Tracker

The tracker coordinates communication among peers.

#### Responsibilities

- Maintains list of active peers
- Helps peers locate file sources

#### Important Note

Tracker does not store the actual multimedia file.

### 3. Shared Multimedia File

Large multimedia files are divided into:

- Small blocks/pieces

Example:

```
Movie File
 ├── Piece 1
 ├── Piece 2
 ├── Piece 3
 └── Piece N
```

### 4. Metadata File

Contains information about:

- File name
- File size
- Piece size
- Tracker address

In BitTorrent, this is the:

```
.torrent file
```

## Features of P2P Multimedia Systems

1. Decentralized sharing
2. Scalability
3. Parallel downloading
4. Fault tolerance
5. Efficient bandwidth utilization

## Advantages of P2P Multimedia Sharing

### 1. Scalability

More users increase available upload capacity.

### 2. Reduced Server Load

Content distributed among peers instead of one server.

### 3. Faster Downloads

Different file pieces downloaded simultaneously from multiple peers.

### 4. Fault Tolerance

Failure of one peer does not stop downloads.

### 5. Efficient Multimedia Distribution

Suitable for large video and audio files.

## Challenges in P2P Systems

### 1. Security Risks

Malicious peers may distribute harmful files.

### 2. Free Riding

Some users download without uploading.

### 3. Peer Availability

Peers may disconnect frequently.

### 4. Copyright Issues

Unauthorized sharing of copyrighted multimedia.

## BitTorrent Protocol

### Introduction

BitTorrent is a popular P2P protocol designed for efficient distribution of large files over the internet.

It minimizes:

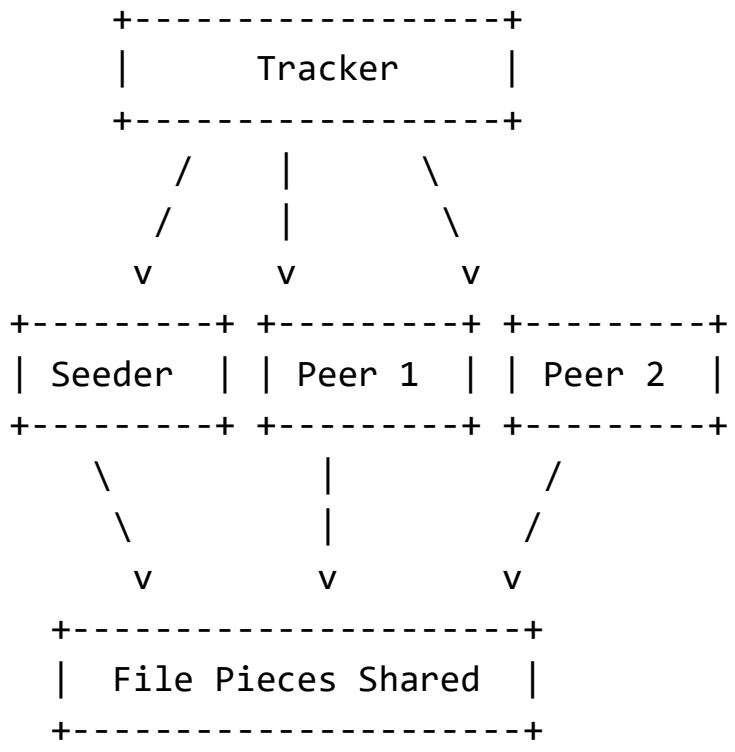
- Server bandwidth usage
- Download bottlenecks

by distributing data among participating peers.

## Important Terms in BitTorrent

| Term | Meaning |
|---|---|
| Torrent | Metadata file describing shared content |
| Peer | Participant downloading/uploading |
| Seeder | Peer with complete file |
| Leecher | Peer still downloading |
| Swarm | Group of peers sharing same file |
| Tracker | Coordinates peer discovery |

## BitTorrent Architecture Diagram



## Operation of BitTorrent Protocol

### Step 1: Torrent File Creation

A small `.torrent` file is created containing:

- File information
- Tracker address
- Hash values of pieces

### Step 2: Torrent Download

User downloads the `.torrent` file.

### Step 3: Contact Tracker

BitTorrent client contacts the tracker.

Tracker returns:

- List of active peers

### Step 4: Join Swarm

The peer joins the swarm sharing that file

### Step 5: Piece Downloading

The file is divided into pieces.

The peer:

- Downloads different pieces from multiple peers simultaneously.

Example:

```
Piece 1 ← Peer A
Piece 2 ← Peer B
Piece 3 ← Peer C
```

### Step 6: Piece Uploading

As soon as a peer downloads a piece:

- It begins uploading that piece to others.

This creates cooperative sharing.

### Step 7: File Completion

After all pieces are received:

- Peer reconstructs complete multimedia file.

The peer may continue as a:

Seeder

### Piece Selection Strategy

BitTorrent uses intelligent piece selection.

### Rarest First Algorithm

Peers download the rarest available pieces first.

### Advantages

- Improves availability
- Prevents missing pieces

### Choking and Unchoking

BitTorrent encourages fair sharing.

### Choking

Peer temporarily refuses upload.

### Unchoking

Peer allows uploads to cooperative peers.

This discourages free riding.

## Advantages of BitTorrent

### 1. High Scalability

Performance improves as peers increase.

### 2. Efficient Bandwidth Usage

Upload load distributed among peers.

### 3. Faster Downloads

Parallel downloads from multiple sources.

### 4. Reliability

Multiple sources provide redundancy.

### 5. Reduced Server Cost

Minimal central server involvement.

## Limitations of BitTorrent

### 1. Dependence on Seeders

Low seeders reduce availability.

### 2. Security Concerns

Possibility of malicious or fake files.

### 3. Legal Issues

Can be used for unauthorized content distribution.

### 4. Variable Performance

Depends on peer availability and bandwidth.

## Applications of BitTorrent

- Multimedia distribution
- Software distribution
- Linux ISO downloads
- Game updates
- Large scientific datasets

## Conclusion

Peer-to-peer download systems efficiently support large multimedia file sharing by distributing data transfer among multiple peers. BitTorrent enhances scalability, reliability, and bandwidth utilization by dividing files into pieces and enabling cooperative downloading and uploading among peers, making it one of the most effective protocols for large-scale multimedia distribution.